Statistics

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Basic Statistics 1

2 C Int Lists 11

3 Time Series 17

4 Hidden Markov Models 49

5 Continuous Emission Hidden Markov Models 65

6 Distributions used in implementing Hidden Markov Models 81

7 Hidden Markov Models – Utility functions 89

8 Discrete Gaussian Samplers over the Integers 93

9 Discrete Gaussian Samplers for $\mathbf{Z}[x]$ 101

10 Discrete Gaussian Samplers over Lattices 105

11 T-test using R 117

12 Indices and Tables 119

Python Module Index 121

Index 123

CHAPTER

ONE

BASIC STATISTICS

This file contains basic descriptive functions. Included are the mean, median, mode, moving average, standard deviation, and the variance. When calling a function on data, there are checks for functions already defined for that data type.

The *mean()* function returns the arithmetic mean (the sum of all the members of a list, divided by the number of members). Further revisions may include the geometric and harmonic mean. The *median()* function returns the number separating the higher half of a sample from the lower half. The *mode()* returns the most common occurring member of a sample, plus the number of times it occurs. If entries occur equally common, the smallest of a list of the most common entries is returned. The *moving_average()* is a finite impulse response filter, creating a series of averages using a user-defined number of subsets of the full data set. The *std()* and the *variance()* return a measurement of how far data points tend to be from the arithmetic mean.

Functions are available in the namespace `stats`, i.e. you can use them by typing `stats.mean`, `stats.median`, etc.

REMARK: If all the data you are working with are floating point numbers, you may find *TimeSeries* helpful, since it is extremely fast and offers many of the same descriptive statistics as in the module.

AUTHOR:

- Andrew Hou (11/06/2009)

`sage.stats.basic_stats.mean(v)`

Return the mean of the elements of v.

We define the mean of the empty list to be the (symbolic) NaN, following the convention of MATLAB, Scipy, and R.

This function is deprecated. Use `numpy.mean()` or `numpy.nanmean()` instead.

INPUT:

- `v` – list of numbers

OUTPUT: a number

EXAMPLES:

```
sage: mean([pi, e])
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mean is deprecated;
use numpy.mean or numpy.nanmean instead
See https://github.com/sagemath/sage/issues/29662 for details.
1/2*pi + 1/2*e
sage: mean([])
NaN
sage: mean([I, sqrt(2), 3/5])
1/3*sqrt(2) + 1/3*I + 1/5
```

(continues on next page)

(continued from previous page)

```
sage: mean([RIF(1.0103,1.0103), RIF(2)])
1.5051500000000000?
sage: mean(range(4))
3/2
sage: v = stats.TimeSeries([1..100])                                    #␣
↪needs numpy
sage: mean(v)                                                           #␣
↪needs numpy
50.5
```

```
>>> from sage.all import *
>>> mean([pi, e])
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mean is deprecated;
use numpy.mean or numpy.nanmean instead
See https://github.com/sagemath/sage/issues/29662 for details.
1/2*pi + 1/2*e
>>> mean([])
NaN
>>> mean([I, sqrt(Integer(2)), Integer(3)/Integer(5)])
1/3*sqrt(2) + 1/3*I + 1/5
>>> mean([RIF(RealNumber('1.0103'),RealNumber('1.0103')), RIF(Integer(2))])
1.5051500000000000?
>>> mean(range(Integer(4)))
3/2
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(100))))  ␣
↪                                        # needs numpy
>>> mean(v)                                                             #␣
↪needs numpy
50.5
```

sage.stats.basic_stats.**median**(*v*)

Return the median (middle value) of the elements of v.

If v is empty, we define the median to be NaN, which is consistent with NumPy (note that R returns NULL). If v is comprised of strings, `TypeError` occurs. For elements other than numbers, the median is a result of `sorted()`.

This function is deprecated. Use `numpy.median()` or `numpy.nanmedian()` instead.

INPUT:

- `v` – list

OUTPUT: median element of v

EXAMPLES:

```
sage: median([1,2,3,4,5])
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.median is deprecated;
use numpy.median or numpy.nanmedian instead
See https://github.com/sagemath/sage/issues/29662 for details.
3
sage: median([e, pi])
1/2*pi + 1/2*e
```

(continues on next page)

(continued from previous page)

```
sage: median(['sage', 'linux', 'python'])
'python'
sage: median([])
NaN
sage: class MyClass:
....:     def median(self):
....:         return 1
sage: stats.median(MyClass())
1
```

```
>>> from sage.all import *
>>> median([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.median is deprecated;
use numpy.median or numpy.nanmedian instead
See https://github.com/sagemath/sage/issues/29662 for details.
3
>>> median([e, pi])
1/2*pi + 1/2*e
>>> median(['sage', 'linux', 'python'])
'python'
>>> median([])
NaN
>>> class MyClass:
...     def median(self):
...         return Integer(1)
>>> stats.median(MyClass())
1
```

sage.stats.basic_stats.**mode**(*v*)

Return the mode of v.

The mode is the list of the most frequently occurring elements in v. If n is the most times that any element occurs in v, then the mode is the list of elements of v that occur n times. The list is sorted if possible.

This function is deprecated. Use `scipy.stats.mode()` or `statistics.mode()` instead.

Note

The elements of v must be hashable.

INPUT:

- `v` – list

OUTPUT: list (sorted if possible)

EXAMPLES:

```
sage: v = [1,2,4,1,6,2,6,7,1]
sage: mode(v)
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mode is deprecated;
```

(continues on next page)

(continued from previous page)

```
use scipy.stats.mode or statistics.mode instead
See https://github.com/sagemath/sage/issues/29662 for details.
[1]
sage: v.count(1)
3
sage: mode([])
[]

sage: mode([1,2,3,4,5])
[1, 2, 3, 4, 5]
sage: mode([3,1,2,1,2,3])
[1, 2, 3]
sage: mode([0, 2, 7, 7, 13, 20, 2, 13])
[2, 7, 13]

sage: mode(['sage', 'four', 'I', 'three', 'sage', 'pi'])
['sage']

sage: class MyClass:
....:   def mode(self):
....:       return [1]
sage: stats.mode(MyClass())
[1]
```

```
>>> from sage.all import *
>>> v = [Integer(1),Integer(2),Integer(4),Integer(1),Integer(6),Integer(2),
↪Integer(6),Integer(7),Integer(1)]
>>> mode(v)
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mode is deprecated;
use scipy.stats.mode or statistics.mode instead
See https://github.com/sagemath/sage/issues/29662 for details.
[1]
>>> v.count(Integer(1))
3
>>> mode([])
[]

>>> mode([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
[1, 2, 3, 4, 5]
>>> mode([Integer(3),Integer(1),Integer(2),Integer(1),Integer(2),Integer(3)])
[1, 2, 3]
>>> mode([Integer(0), Integer(2), Integer(7), Integer(7), Integer(13),␣
↪Integer(20), Integer(2), Integer(13)])
[2, 7, 13]

>>> mode(['sage', 'four', 'I', 'three', 'sage', 'pi'])
['sage']

>>> class MyClass:
...   def mode(self):
...       return [Integer(1)]
```

(continues on next page)

(continued from previous page)

```
>>> stats.mode(MyClass())
[1]
```

sage.stats.basic_stats.**moving_average**(*v*, *n*)

Return the moving average of a list v.

The moving average of a list is often used to smooth out noisy data.

If v is empty, we define the entries of the moving average to be NaN.

This method is deprecated. Use `pandas.Series.rolling` instead.

INPUT:

- `v` – list
- `n` – the number of values used in computing each average

OUTPUT: list of length `len(v)-n+1`, since we do not fabric any values

EXAMPLES:

```
sage: moving_average([1..10], 1)
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.moving_average is deprecated;
use pandas.Series.rolling instead
See https://github.com/sagemath/sage/issues/29662 for details.
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sage: moving_average([1..10], 4)
[5/2, 7/2, 9/2, 11/2, 13/2, 15/2, 17/2]
sage: moving_average([], 1)
[]
sage: moving_average([pi, e, I, sqrt(2), 3/5], 2)
[1/2*pi + 1/2*e, 1/2*e + 1/2*I, 1/2*sqrt(2) + 1/2*I,
 1/2*sqrt(2) + 3/10]
```

```
>>> from sage.all import *
>>> moving_average((ellipsis_range(Integer(1),Ellipsis,Integer(10))), Integer(1))
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.moving_average is deprecated;
use pandas.Series.rolling instead
See https://github.com/sagemath/sage/issues/29662 for details.
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> moving_average((ellipsis_range(Integer(1),Ellipsis,Integer(10))), Integer(4))
[5/2, 7/2, 9/2, 11/2, 13/2, 15/2, 17/2]
>>> moving_average([], Integer(1))
[]
>>> moving_average([pi, e, I, sqrt(Integer(2)), Integer(3)/Integer(5)],␣
↪Integer(2))
[1/2*pi + 1/2*e, 1/2*e + 1/2*I, 1/2*sqrt(2) + 1/2*I,
 1/2*sqrt(2) + 3/10]
```

We check if the input is a time series, and if so use the optimized *simple_moving_average()* method, but with (slightly different) meaning as defined above (the point is that *simple_moving_average()* on time series returns n values:

```
sage: a = stats.TimeSeries([1..10])                                    #␣
↪needs numpy
sage: stats.moving_average(a, 3)                                       #␣
↪needs numpy
[2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000]
sage: stats.moving_average(list(a), 3)                                 #␣
↪needs numpy
[2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0]
```

```
>>> from sage.all import *
>>> a = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10))))  ␣
↪                                        # needs numpy
>>> stats.moving_average(a, Integer(3))                                 ␣
↪     # needs numpy
[2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000]
>>> stats.moving_average(list(a), Integer(3))                           ␣
↪     # needs numpy
[2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0]
```

sage.stats.basic_stats.**std**(*v*, *bias=False*)

Return the standard deviation of the elements of v.

We define the standard deviation of the empty list to be NaN, following the convention of MATLAB, Scipy, and R.

This function is deprecated. Use `numpy.std()` or `numpy.nanstd()` instead.

INPUT:

- `v` – list of numbers
- `bias` – boolean (default: `False`); if `False`, divide by `len(v) - 1` instead of `len(v)` to give a less biased estimator (sample) for the standard deviation.

OUTPUT: a number

EXAMPLES:

```
sage: std([1..6], bias=True)
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.std is deprecated;
use numpy.std or numpy.nanstd instead
See https://github.com/sagemath/sage/issues/29662 for details.
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.variance is deprecated;
use numpy.var or numpy.nanvar instead
See https://github.com/sagemath/sage/issues/29662 for details.
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mean is deprecated;
use numpy.mean or numpy.nanmean instead
See https://github.com/sagemath/sage/issues/29662 for details.
1/2*sqrt(35/3)
sage: std([1..6], bias=False)
sqrt(7/2)
sage: std([e, pi])
sqrt(1/2)*abs(pi - e)
```

(continues on next page)

(continued from previous page)

```
sage: std([])
NaN
sage: std([I, sqrt(2), 3/5])
1/15*sqrt(1/2)*sqrt((10*sqrt(2) - 5*I - 3)^2
+ (5*sqrt(2) - 10*I + 3)^2 + (5*sqrt(2) + 5*I - 6)^2)
sage: std([RIF(1.0103, 1.0103), RIF(2)])
0.6998235813403261?

sage: # needs numpy
sage: import numpy
sage: if int(numpy.version.short_version[0]) > 1:
....:     _ = numpy.set_printoptions(legacy="1.25")
sage: x = numpy.array([1,2,3,4,5])
sage: std(x, bias=False)
1.5811388300841898
sage: x = stats.TimeSeries([1..100])
sage: std(x)
29.011491975882016
```

```
>>> from sage.all import *
>>> std((ellipsis_range(Integer(1),Ellipsis,Integer(6))), bias=True)
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.std is deprecated;
use numpy.std or numpy.nanstd instead
See https://github.com/sagemath/sage/issues/29662 for details.
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.variance is deprecated;
use numpy.var or numpy.nanvar instead
See https://github.com/sagemath/sage/issues/29662 for details.
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.mean is deprecated;
use numpy.mean or numpy.nanmean instead
See https://github.com/sagemath/sage/issues/29662 for details.
1/2*sqrt(35/3)
>>> std((ellipsis_range(Integer(1),Ellipsis,Integer(6))), bias=False)
sqrt(7/2)
>>> std([e, pi])
sqrt(1/2)*abs(pi - e)
>>> std([])
NaN
>>> std([I, sqrt(Integer(2)), Integer(3)/Integer(5)])
1/15*sqrt(1/2)*sqrt((10*sqrt(2) - 5*I - 3)^2
+ (5*sqrt(2) - 10*I + 3)^2 + (5*sqrt(2) + 5*I - 6)^2)
>>> std([RIF(RealNumber('1.0103'), RealNumber('1.0103')), RIF(Integer(2))])
0.6998235813403261?

>>> # needs numpy
>>> import numpy
>>> if int(numpy.version.short_version[Integer(0)]) > Integer(1):
...     _ = numpy.set_printoptions(legacy="1.25")
>>> x = numpy.array([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
>>> std(x, bias=False)
```

(continues on next page)

(continued from previous page)

```
1.5811388300841898
>>> x = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(100))))
>>> std(x)
29.011491975882016
```

sage.stats.basic_stats.**variance**(*v*, *bias=False*)

Return the variance of the elements of v.

We define the variance of the empty list to be NaN, following the convention of MATLAB, Scipy, and R.

This function is deprecated. Use `numpy.var()` or `numpy.nanvar()` instead.

INPUT:

- `v` – list of numbers
- `bias` – boolean (default: `False`); if `False`, divide by `len(v) - 1` instead of `len(v)` to give a less biased estimator (sample) for the standard deviation.

OUTPUT: a number

EXAMPLES:

```
sage: variance([1..6])
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.variance is deprecated;
use numpy.var or numpy.nanvar instead
See https://github.com/sagemath/sage/issues/29662 for details.
7/2
sage: variance([1..6], bias=True)
35/12
sage: variance([e, pi])
1/2*(pi - e)^2
sage: variance([])
NaN
sage: variance([I, sqrt(2), 3/5])
1/450*(10*sqrt(2) - 5*I - 3)^2 + 1/450*(5*sqrt(2) - 10*I + 3)^2
+ 1/450*(5*sqrt(2) + 5*I - 6)^2
sage: variance([RIF(1.0103, 1.0103), RIF(2)])
0.48975304500000000?
sage: import numpy                                                          #␣
↪needs numpy
sage: if int(numpy.version.short_version[0]) > 1:                           #␣
↪needs numpy
....:     _ = numpy.set_printoptions(legacy="1.25")                          ␣
↪  # needs numpy
sage: x = numpy.array([1,2,3,4,5])                                          #␣
↪needs numpy
sage: variance(x, bias=False)                                               #␣
↪needs numpy
2.5
sage: x = stats.TimeSeries([1..100])
sage: variance(x)
841.6666666666666
sage: variance(x, bias=True)
```

(continues on next page)

(continued from previous page)

```
833.25
sage: class MyClass:
....:   def variance(self, bias=False):
....:       return 1
sage: stats.variance(MyClass())
1
sage: class SillyPythonList:
....:   def __init__(self):
....:       self.__list = [2, 4]
....:   def __len__(self):
....:       return len(self.__list)
....:   def __iter__(self):
....:       return self.__list.__iter__()
....:   def mean(self):
....:       return 3
sage: R = SillyPythonList()
sage: variance(R)
2
sage: variance(R, bias=True)
1
```

```
>>> from sage.all import *
>>> variance((ellipsis_range(Integer(1),Ellipsis,Integer(6))))
doctest:warning...
DeprecationWarning: sage.stats.basic_stats.variance is deprecated;
use numpy.var or numpy.nanvar instead
See https://github.com/sagemath/sage/issues/29662 for details.
7/2
>>> variance((ellipsis_range(Integer(1),Ellipsis,Integer(6))), bias=True)
35/12
>>> variance([e, pi])
1/2*(pi - e)^2
>>> variance([])
NaN
>>> variance([I, sqrt(Integer(2)), Integer(3)/Integer(5)])
1/450*(10*sqrt(2) - 5*I - 3)^2 + 1/450*(5*sqrt(2) - 10*I + 3)^2
+ 1/450*(5*sqrt(2) + 5*I - 6)^2
>>> variance([RIF(RealNumber('1.0103'), RealNumber('1.0103')), RIF(Integer(2))])
0.4897530450000000?
>>> import numpy                                                    #␣
↪needs numpy
>>> if int(numpy.version.short_version[Integer(0)]) > Integer(1):          ␣
↪              # needs numpy
...     _ = numpy.set_printoptions(legacy="1.25")
↪# needs numpy
>>> x = numpy.array([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])    ␣
↪                                        # needs numpy
>>> variance(x, bias=False)                                         #␣
↪needs numpy
2.5
>>> x = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(100))))
>>> variance(x)
```

(continues on next page)

(continued from previous page)

```
841.6666666666666
>>> variance(x, bias=True)
833.25
>>> class MyClass:
...   def variance(self, bias=False):
...       return Integer(1)
>>> stats.variance(MyClass())
1
>>> class SillyPythonList:
...   def __init__(self):
...       self.__list = [Integer(2), Integer(4)]
...   def __len__(self):
...       return len(self.__list)
...   def __iter__(self):
...       return self.__list.__iter__()
...   def mean(self):
...       return Integer(3)
>>> R = SillyPythonList()
>>> variance(R)
2
>>> variance(R, bias=True)
1
```

CHAPTER

TWO

C INT LISTS

This is a class for fast basic operations with lists of C ints. It is similar to the double precision TimeSeries class. It has all the standard C int semantics, of course, including overflow. It is also similar to the Python list class, except all elements are C ints, which makes some operations much, much faster. For example, concatenating two IntLists can be over 10 times faster than concatenating the corresponding Python lists of ints, and taking slices is also much faster.

AUTHOR:

- William Stein, 2010-03

class sage.stats.intlist.**IntList**

Bases: object

A list of C int's.

list()

Return Python list version of `self` with Python ints as entries.

EXAMPLES:

```
sage: a = stats.IntList([1..15]); a
[1, 2, 3, 4, 5 ... 11, 12, 13, 14, 15]
sage: a.list()
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
sage: list(a) == a.list()
True
sage: type(a.list()[0])
<... 'int'>
```

```
>>> from sage.all import *
>>> a = stats.IntList((ellipsis_range(Integer(1),Ellipsis,Integer(15)))); a
[1, 2, 3, 4, 5 ... 11, 12, 13, 14, 15]
>>> a.list()
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
>>> list(a) == a.list()
True
>>> type(a.list()[Integer(0)])
<... 'int'>
```

max(*index=False*)

Return the largest value in this time series. If this series has length 0 we raise a `ValueError`

INPUT:

- `index` – boolean (default: `False`); if `True`, also return index of maximum entry

OUTPUT:

- integer – largest value
- integer – index of largest value; only returned if `index=True`

EXAMPLES:

```
sage: v = stats.IntList([1,-4,3,-2,-4,3])
sage: v.max()
3
sage: v.max(index=True)
(3, 2)
```

```
>>> from sage.all import *
>>> v = stats.IntList([Integer(1),-Integer(4),Integer(3),-Integer(2),-
↪Integer(4),Integer(3)])
>>> v.max()
3
>>> v.max(index=True)
(3, 2)
```

min(*index=False*)

Return the smallest value in this integer list. If this series has length 0 we raise a `ValueError`.

INPUT:

- `index` – boolean (default: `False`); if `True`, also return index of minimal entry

OUTPUT:

- `float` – smallest value
- `integer` – index of smallest value; only returned if `index=True`

EXAMPLES:

```
sage: v = stats.IntList([1,-4,3,-2,-4])
sage: v.min()
-4
sage: v.min(index=True)
(-4, 1)
```

```
>>> from sage.all import *
>>> v = stats.IntList([Integer(1),-Integer(4),Integer(3),-Integer(2),-
↪Integer(4)])
>>> v.min()
-4
>>> v.min(index=True)
(-4, 1)
```

plot(**args*, ***kwds*)

Return a plot of this *IntList*.

This just constructs the corresponding double-precision floating point *TimeSeries* object, passing on all arguments.

EXAMPLES:

```
sage: stats.IntList([3,7,19,-2]).plot()
Graphics object consisting of 1 graphics primitive
sage: stats.IntList([3,7,19,-2]).plot(color='red',
....:                                 pointsize=50, points=True)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> stats.IntList([Integer(3),Integer(7),Integer(19),-Integer(2)]).plot()
Graphics object consisting of 1 graphics primitive
>>> stats.IntList([Integer(3),Integer(7),Integer(19),-Integer(2)]).plot(color=
↪'red',
...                                 pointsize=Integer(50), points=True)
Graphics object consisting of 1 graphics primitive
```

plot_histogram(**args*, ***kwds*)

Return a histogram plot of this *IntList*.

This just constructs the corresponding double-precision floating point *TimeSeries* object, and plots it, passing on all arguments.

EXAMPLES:

```
sage: stats.IntList([1..15]).plot_histogram()
Graphics object consisting of 50 graphics primitives
```

```
>>> from sage.all import *
>>> stats.IntList((ellipsis_range(Integer(1),Ellipsis,Integer(15)))).plot_
↪histogram()
Graphics object consisting of 50 graphics primitives
```

prod()

Return the product of the entries of `self`.

EXAMPLES:

```
sage: a = stats.IntList([1..10]); a
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sage: a.prod()
3628800
sage: factorial(10)
3628800
```

```
>>> from sage.all import *
>>> a = stats.IntList((ellipsis_range(Integer(1),Ellipsis,Integer(10)))); a
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> a.prod()
3628800
>>> factorial(Integer(10))
3628800
```

Note that there can be overflow:

```
sage: a = stats.IntList([2^30, 2]); a
[1073741824, 2]
sage: a.prod()
-2147483648
```

```
>>> from sage.all import *
>>> a = stats.IntList([Integer(2)**Integer(30), Integer(2)]); a
[1073741824, 2]
>>> a.prod()
-2147483648
```

sum()

Return the sum of the entries of `self`.

EXAMPLES:

```
sage: stats.IntList([1..100]).sum()
5050
```

```
>>> from sage.all import *
>>> stats.IntList((ellipsis_range(Integer(1),Ellipsis,Integer(100)))).sum()
5050
```

Note that there can be overflow, since the entries are C ints:

```
sage: a = stats.IntList([2^30,2^30]); a
[1073741824, 1073741824]
sage: a.sum()
-2147483648
```

```
>>> from sage.all import *
>>> a = stats.IntList([Integer(2)**Integer(30),Integer(2)**Integer(30)]); a
[1073741824, 1073741824]
>>> a.sum()
-2147483648
```

time_series()

Return *TimeSeries* version of `self`, which involves changing each entry to a double.

EXAMPLES:

```
sage: T = stats.IntList([-2,3,5]).time_series(); T
[-2.0000, 3.0000, 5.0000]
sage: type(T)
<... 'sage.stats.time_series.TimeSeries'>
```

```
>>> from sage.all import *
>>> T = stats.IntList([-Integer(2),Integer(3),Integer(5)]).time_series(); T
[-2.0000, 3.0000, 5.0000]
>>> type(T)
<... 'sage.stats.time_series.TimeSeries'>
```

sage.stats.intlist.**unpickle_intlist_v1**(*v*, *n*)

Version 1 unpickle method.

INPUT:

- v – a raw char buffer

EXAMPLES:

```
sage: v = stats.IntList([1,2,3])
sage: s = v.__reduce__()[1][0]
sage: type(s) == type(b'')
True
sage: sage.stats.intlist.unpickle_intlist_v1(s, 3)
[1, 2, 3]
sage: sage.stats.intlist.unpickle_intlist_v1(s+s,6)
[1, 2, 3, 1, 2, 3]
sage: sage.stats.intlist.unpickle_intlist_v1(b'',0)
[]
```

```
>>> from sage.all import *
>>> v = stats.IntList([Integer(1),Integer(2),Integer(3)])
>>> s = v.__reduce__()[Integer(1)][Integer(0)]
>>> type(s) == type(b'')
True
>>> sage.stats.intlist.unpickle_intlist_v1(s, Integer(3))
[1, 2, 3]
>>> sage.stats.intlist.unpickle_intlist_v1(s+s,Integer(6))
[1, 2, 3, 1, 2, 3]
>>> sage.stats.intlist.unpickle_intlist_v1(b'',Integer(0))
[]
```

CHAPTER

THREE

TIME SERIES

This is a module for working with discrete floating point time series. It is designed so that every operation is very fast, typically much faster than with other generic code, e.g., Python lists of doubles or even NumPy arrays. The semantics of time series is more similar to Python lists of doubles than Sage real double vectors or NumPy 1-D arrays. In particular, time series are not endowed with much algebraic structure and are always mutable.

Note

NumPy arrays are faster at slicing, since slices return references, and NumPy arrays have strides. However, this speed at slicing makes NumPy slower at certain other operations.

EXAMPLES:

```
sage: set_random_seed(1)
sage: t = stats.TimeSeries([random()-0.5 for _ in range(10)]); t
[0.3294, 0.0959, -0.0706, -0.4646, 0.4311, 0.2275, -0.3840, -0.3528, -0.4119, -0.2933]
sage: t.sums()
[0.3294, 0.4253, 0.3547, -0.1099, 0.3212, 0.5487, 0.1647, -0.1882, -0.6001, -0.8933]
sage: t.exponential_moving_average(0.7)
[0.0000, 0.3294, 0.1660, 0.0003, -0.3251, 0.2042, 0.2205, -0.2027, -0.3078, -0.3807]
sage: t.standard_deviation()
0.33729638212891383
sage: t.mean()
-0.08933425506929439
sage: t.variance()
0.1137688493972542...
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(1))
>>> t = stats.TimeSeries([random()-RealNumber('0.5') for _ in range(Integer(10))]); t
[0.3294, 0.0959, -0.0706, -0.4646, 0.4311, 0.2275, -0.3840, -0.3528, -0.4119, -0.2933]
>>> t.sums()
[0.3294, 0.4253, 0.3547, -0.1099, 0.3212, 0.5487, 0.1647, -0.1882, -0.6001, -0.8933]
>>> t.exponential_moving_average(RealNumber('0.7'))
[0.0000, 0.3294, 0.1660, 0.0003, -0.3251, 0.2042, 0.2205, -0.2027, -0.3078, -0.3807]
>>> t.standard_deviation()
0.33729638212891383
>>> t.mean()
-0.08933425506929439
>>> t.variance()
```

(continues on next page)

(continued from previous page)

```
0.1137688493972542...
```

AUTHOR:

- William Stein

class sage.stats.time_series.**TimeSeries**

Bases: object

Initialize new time series.

INPUT:

- values – integer (number of values) or an iterable of floats
- initialize – boolean (default: True); if False, do not bother to zero out the entries of the new time series. For large series that you are going to just fill in, this can be way faster.

EXAMPLES:

This implicitly calls init:

```
sage: stats.TimeSeries([pi, 3, 18.2])
[3.1416, 3.0000, 18.2000]
```

```
>>> from sage.all import *
>>> stats.TimeSeries([pi, Integer(3), RealNumber('18.2')])
[3.1416, 3.0000, 18.2000]
```

Conversion from a NumPy 1-D array, which is very fast:

```
sage: v = stats.TimeSeries([1..5])
sage: w = v.numpy()
sage: stats.TimeSeries(w)
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(5))))
>>> w = v.numpy()
>>> stats.TimeSeries(w)
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
```

Conversion from an n-dimensional NumPy array also works:

```
sage: import numpy
sage: v = numpy.array([[1,2], [3,4]], dtype=float); v
array([[1., 2.],
       [3., 4.]])
sage: stats.TimeSeries(v)
[1.0000, 2.0000, 3.0000, 4.0000]
sage: stats.TimeSeries(v[:,0])
[1.0000, 3.0000]
sage: u = numpy.array([[1,2],[3,4]])
sage: stats.TimeSeries(u)
[1.0000, 2.0000, 3.0000, 4.0000]
```

```
>>> from sage.all import *
>>> import numpy
>>> v = numpy.array([[Integer(1),Integer(2)], [Integer(3),Integer(4)]],␣
↪dtype=float); v
array([[1., 2.],
       [3., 4.]])
>>> stats.TimeSeries(v)
[1.0000, 2.0000, 3.0000, 4.0000]
>>> stats.TimeSeries(v[:,Integer(0)])
[1.0000, 3.0000]
>>> u = numpy.array([[Integer(1),Integer(2)],[Integer(3),Integer(4)]])
>>> stats.TimeSeries(u)
[1.0000, 2.0000, 3.0000, 4.0000]
```

For speed purposes we don't initialize (so value is garbage):

```
sage: t = stats.TimeSeries(10, initialize=False)
```

```
>>> from sage.all import *
>>> t = stats.TimeSeries(Integer(10), initialize=False)
```

abs()

Return new time series got by replacing all entries of `self` by their absolute value.

OUTPUT: a new time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1,3.1908439,-4,5.93932])
sage: v
[1.0000, 3.1908, -4.0000, 5.9393]
sage: v.abs()
[1.0000, 3.1908, 4.0000, 5.9393]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),RealNumber('3.1908439'),-Integer(4),
↪RealNumber('5.93932')])
>>> v
[1.0000, 3.1908, -4.0000, 5.9393]
>>> v.abs()
[1.0000, 3.1908, 4.0000, 5.9393]
```

add_entries(*t*)

Add corresponding entries of `self` and `t` together, extending either `self` or `t` by 0s if they do not have the same length.

> **Note**
>
> To add a single number to the entries of a time series, use the `add_scalar` method.

INPUT:

- `t` – a time series

OUTPUT:

A time series with length the maximum of the lengths of `self` and `t`.

EXAMPLES:

```
sage: v = stats.TimeSeries([1,2,-5]); v
[1.0000, 2.0000, -5.0000]
sage: v.add_entries([3,4])
[4.0000, 6.0000, -5.0000]
sage: v.add_entries(v)
[2.0000, 4.0000, -10.0000]
sage: v.add_entries([3,4,7,18.5])
[4.0000, 6.0000, 2.0000, 18.5000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(2),-Integer(5)]); v
[1.0000, 2.0000, -5.0000]
>>> v.add_entries([Integer(3),Integer(4)])
[4.0000, 6.0000, -5.0000]
>>> v.add_entries(v)
[2.0000, 4.0000, -10.0000]
>>> v.add_entries([Integer(3),Integer(4),Integer(7),RealNumber('18.5')])
[4.0000, 6.0000, 2.0000, 18.5000]
```

add_scalar(*s*)

Return new time series obtained by adding a scalar to every value in the series.

> **Note**
>
> To add componentwise, use the *add_entries()* method.

INPUT:

- `s` – a float

OUTPUT: a new time series with `s` added to all values

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.add_scalar(0.5)
[5.5000, 4.5000, 1.8000, 2.5000, 8.5000, 10.5000, 3.5000, -4.5000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.add_scalar(RealNumber('0.5'))
[5.5000, 4.5000, 1.8000, 2.5000, 8.5000, 10.5000, 3.5000, -4.5000]
```

autocorrelation(*k=1*)

Return the k-th sample autocorrelation of this time series x_i.

Let μ be the sample mean. Then the sample autocorrelation function is

$$\frac{\sum_{t=0}^{n-k-1}(x_t-\mu)(x_{t+k}-\mu)}{\sum_{t=0}^{n-1}(x_t-\mu)^2}.$$

Note that the variance must be nonzero or you will get a `ZeroDivisionError`.

INPUT:

- `k` – nonnegative integer (default: 1)

OUTPUT: a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([13,8,15,4,4,12,11,7,14,12])
sage: v.autocorrelation()
-0.1875
sage: v.autocorrelation(1)
-0.1875
sage: v.autocorrelation(0)
1.0
sage: v.autocorrelation(2)
-0.20138888888888887
sage: v.autocorrelation(3)
0.18055555555555555

sage: stats.TimeSeries([1..1000]).autocorrelation()
0.997
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(13),Integer(8),Integer(15),Integer(4),
↪Integer(4),Integer(12),Integer(11),Integer(7),Integer(14),Integer(12)])
>>> v.autocorrelation()
-0.1875
>>> v.autocorrelation(Integer(1))
-0.1875
>>> v.autocorrelation(Integer(0))
1.0
>>> v.autocorrelation(Integer(2))
-0.20138888888888887
>>> v.autocorrelation(Integer(3))
0.18055555555555555

>>> stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(1000)))).
↪autocorrelation()
0.997
```

autocovariance(*k=0*)

Return the k-th autocovariance function $\gamma(k)$ of `self`.

This is the covariance of `self` with `self` shifted by k, i.e.,

$$\left(\sum_{t=0}^{n-k-1}(x_t-\mu)(x_{t+k}-\mu)\right)\Big/ n,$$

where n is the length of `self`.

Note the denominator of n, which gives a "better" sample estimator.

INPUT:

- `k` – nonnegative integer (default: 0)

OUTPUT: float

EXAMPLES:

```
sage: v = stats.TimeSeries([13,8,15,4,4,12,11,7,14,12])
sage: v.autocovariance(0)
14.4
sage: mu = v.mean(); sum([(a-mu)^2 for a in v])/len(v)
14.4
sage: v.autocovariance(1)
-2.7
sage: mu = v.mean()
sage: sum((v[i]-mu)*(v[i+1]-mu) for i in range(len(v)-1))/len(v)
-2.7
sage: v.autocovariance(1)
-2.7
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(13),Integer(8),Integer(15),Integer(4),
↪Integer(4),Integer(12),Integer(11),Integer(7),Integer(14),Integer(12)])
>>> v.autocovariance(Integer(0))
14.4
>>> mu = v.mean(); sum([(a-mu)**Integer(2) for a in v])/len(v)
14.4
>>> v.autocovariance(Integer(1))
-2.7
>>> mu = v.mean()
>>> sum((v[i]-mu)*(v[i+Integer(1)]-mu) for i in range(len(v)-Integer(1)))/
↪len(v)
-2.7
>>> v.autocovariance(Integer(1))
-2.7
```

We illustrate with a random sample that an independently and identically distributed distribution with zero mean and variance σ^2 has autocovariance function $\gamma(h)$ with $\gamma(0) = \sigma^2$ and $\gamma(h) = 0$ for $h \neq 0$.

```
sage: set_random_seed(0)
sage: v = stats.TimeSeries(10^6)
sage: v.randomize('normal', 0, 5)
[3.3835, -2.0055, 1.7882, -2.9319, -4.6827 ...
 -5.1868, 9.2613, 0.9274, -6.2282, -8.7652]
sage: v.autocovariance(0)
24.95410689...
sage: v.autocovariance(1)
-0.00508390...
sage: v.autocovariance(2)
0.022056325...
sage: v.autocovariance(3)
-0.01902000...
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> v = stats.TimeSeries(Integer(10)**Integer(6))
>>> v.randomize('normal', Integer(0), Integer(5))
[3.3835, -2.0055, 1.7882, -2.9319, -4.6827 ...
 -5.1868, 9.2613, 0.9274, -6.2282, -8.7652]
>>> v.autocovariance(Integer(0))
24.95410689...
>>> v.autocovariance(Integer(1))
-0.00508390...
>>> v.autocovariance(Integer(2))
0.022056325...
>>> v.autocovariance(Integer(3))
-0.01902000...
```

autoregressive_fit(M)

This method fits the time series to an autoregressive process of order M. That is, we assume the process is given by $X_t - \mu = a_1(X_{t-1} - \mu) + a_2(X_{t-1} - \mu) + \cdots + a_M(X_{t-M} - \mu) + Z_t$ where μ is the mean of the process and Z_t is noise. This method returns estimates for $a_1, \ldots, a_M$.

The method works by solving the Yule-Walker equations $\Gamma a = \gamma$, where $\gamma = (\gamma(1), \ldots, \gamma(M))$, $a = (a_1, \ldots, a_M)$ with $\gamma(i)$ the autocovariance of lag i and $\Gamma_{ij} = \gamma(i - j)$.

Warning

The input sequence is assumed to be stationary, which means that the autocovariance $\langle y_j y_k \rangle$ depends only on the difference $|j - k|$.

INPUT:

- M – integer

OUTPUT: a time series – the coefficients of the autoregressive process

EXAMPLES:

```
sage: set_random_seed(0)
sage: v = stats.TimeSeries(10^4).randomize('normal').sums()
sage: F = v.autoregressive_fit(100)
sage: v
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ... 87.6759, 87.6825, 87.4120, 87.
↪6639, 86.3194]
sage: v.autoregressive_forecast(F)
86.0177285042...
sage: F
[1.0148, -0.0029, -0.0105, 0.0067, -0.0232 ... -0.0106, -0.0068, 0.0085, -0.
↪0131, 0.0092]

sage: set_random_seed(0)
sage: t = stats.TimeSeries(2000)
sage: z = stats.TimeSeries(2000)
sage: z.randomize('normal',1)
[1.6767, 0.5989, 1.3576, 0.4136, 0.0635 ... 1.0057, -1.1467, 1.2809, 1.5705,␣
```

(continues on next page)

(continued from previous page)

```
↪1.1095]
sage: t[0]=1
sage: t[1]=2
sage: for i in range(2,2000):
....:     t[i]=t[i-1]-0.5*t[i-2]+z[i]
sage: c=t[0:-1].autoregressive_fit(2)  #recovers recurrence relation
sage: c #should be close to [1,-0.5]
[1.0371, -0.5199]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> v = stats.TimeSeries(Integer(10)**Integer(4)).randomize('normal').sums()
>>> F = v.autoregressive_fit(Integer(100))
>>> v
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ... 87.6759, 87.6825, 87.4120, 87.
↪6639, 86.3194]
>>> v.autoregressive_forecast(F)
86.0177285042...
>>> F
[1.0148, -0.0029, -0.0105, 0.0067, -0.0232 ... -0.0106, -0.0068, 0.0085, -0.
↪0131, 0.0092]

>>> set_random_seed(Integer(0))
>>> t = stats.TimeSeries(Integer(2000))
>>> z = stats.TimeSeries(Integer(2000))
>>> z.randomize('normal',Integer(1))
[1.6767, 0.5989, 1.3576, 0.4136, 0.0635 ... 1.0057, -1.1467, 1.2809, 1.5705,␣
↪1.1095]
>>> t[Integer(0)]=Integer(1)
>>> t[Integer(1)]=Integer(2)
>>> for i in range(Integer(2),Integer(2000)):
...     t[i]=t[i-Integer(1)]-RealNumber('0.5')*t[i-Integer(2)]+z[i]
>>> c=t[Integer(0):-Integer(1)].autoregressive_fit(Integer(2))  #recovers␣
↪recurrence relation
>>> c #should be close to [1,-0.5]
[1.0371, -0.5199]
```

autoregressive_forecast (*filter*)

Given the autoregression coefficients as outputted by the *autoregressive_fit()* command, compute the forecast for the next term in the series.

INPUT:

- `filter` – a time series outputted by the `autoregressive_fit` command

EXAMPLES:

```
sage: set_random_seed(0)
sage: v = stats.TimeSeries(100).randomize('normal').sums()
sage: F = v[:-1].autoregressive_fit(5); F
[1.0019, -0.0524, -0.0643, 0.1323, -0.0539]
sage: v.autoregressive_forecast(F)
11.7820298611...
```

(continues on next page)

(continued from previous page)

```
sage: v
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ... 9.2447, 9.6709, 10.4037, 10.4836,
↪ 12.1960]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> v = stats.TimeSeries(Integer(100)).randomize('normal').sums()
>>> F = v[:-Integer(1)].autoregressive_fit(Integer(5)); F
[1.0019, -0.0524, -0.0643, 0.1323, -0.0539]
>>> v.autoregressive_forecast(F)
11.7820298611...
>>> v
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ... 9.2447, 9.6709, 10.4037, 10.4836,
↪ 12.1960]
```

central_moment(k)

Return the k-th central moment of `self`, which is just the mean of the k-th powers of the differences `self[i] - mu`, where `mu` is the mean of `self`.

INPUT:

- `k` – positive integer

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,2,3])
sage: v.central_moment(2)
0.6666666666666666
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(2),Integer(3)])
>>> v.central_moment(Integer(2))
0.6666666666666666
```

Note that the central moment is different from the moment here, since the mean is not 0:

```
sage: v.moment(2)      # doesn't subtract off mean
4.666666666666667
```

```
>>> from sage.all import *
>>> v.moment(Integer(2))      # doesn't subtract off mean
4.666666666666667
```

We compute the central moment directly:

```
sage: mu = v.mean(); mu
2.0
sage: ((1-mu)^2 + (2-mu)^2 + (3-mu)^2) / 3
0.6666666666666666
```

```
>>> from sage.all import *
>>> mu = v.mean(); mu
2.0
>>> ((Integer(1)-mu)**Integer(2) + (Integer(2)-mu)**Integer(2) + (Integer(3)-
↪mu)**Integer(2)) / Integer(3)
0.666666666666666
```

clip_remove(*min=None*, *max=None*)

Return new time series obtained from `self` by removing all values that are less than or equal to a certain minimum value or greater than or equal to a certain maximum.

INPUT:

- `min` – (default: `None`) `None` or double
- `max` – (default: `None`) `None` or double

OUTPUT: a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1..10])
sage: v.clip_remove(3,7)
[3.0000, 4.0000, 5.0000, 6.0000, 7.0000]
sage: v.clip_remove(3)
[3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.0000]
sage: v.clip_remove(max=7)
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10))))
>>> v.clip_remove(Integer(3),Integer(7))
[3.0000, 4.0000, 5.0000, 6.0000, 7.0000]
>>> v.clip_remove(Integer(3))
[3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.0000]
>>> v.clip_remove(max=Integer(7))
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000]
```

correlation(*other*)

Return the correlation of `self` and `other`, which is the covariance of `self` and `other` divided by the product of their standard deviation.

INPUT:

- `self`, `other` – time series

Whichever time series has more terms is truncated.

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-2,3]); w = stats.TimeSeries([4,5,-10])
sage: v.correlation(w)
-0.558041609...
sage: v.covariance(w)/(v.standard_deviation() * w.standard_deviation())
-0.558041609...
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(2),Integer(3)]); w = stats.
↪TimeSeries([Integer(4),Integer(5),-Integer(10)])
>>> v.correlation(w)
-0.558041609...
>>> v.covariance(w)/(v.standard_deviation() * w.standard_deviation())
-0.558041609...
```

covariance(*other*)

Return the covariance of the time series `self` and `other`.

INPUT:

- `self`, `other` – time series

Whichever time series has more terms is truncated.

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-2,3]); w = stats.TimeSeries([4,5,-10])
sage: v.covariance(w)
-11.777777777777779
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(2),Integer(3)]); w = stats.
↪TimeSeries([Integer(4),Integer(5),-Integer(10)])
>>> v.covariance(w)
-11.777777777777779
```

diffs(*k=1*)

Return the new time series got by taking the differences of successive terms in the time series. So if `self` is the time series $X_0, X_1, X_2, \ldots$, then this function outputs the series $X_1 - X_0, X_2 - X_1, \ldots$. The output series has one less term than the input series. If the optional parameter k is given, return $X_k - X_0, X_{2k} - X_k, \ldots$.

INPUT:

- `k` – positive integer (default: 1)

OUTPUT: a new time series

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000]
sage: v.diffs()
[-1.0000, -2.7000, 0.7000, 6.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000]
>>> v.diffs()
[-1.0000, -2.7000, 0.7000, 6.0000]
```

exp()

Return new time series got by applying the exponential map to all the terms in the time series.

OUTPUT: a new time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1..5]); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
sage: v.exp()
[2.7183, 7.3891, 20.0855, 54.5982, 148.4132]
sage: v.exp().log()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(5)))); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
>>> v.exp()
[2.7183, 7.3891, 20.0855, 54.5982, 148.4132]
>>> v.exp().log()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
```

exponential_moving_average(*alpha*)

Return the exponential moving average time series.

Assumes the input time series was constant with its starting value for negative time. The t-th step of the output is the sum of the previous $k-1$ steps of `self` and the k-th step divided by k.

The 0-th term is formally undefined, so we define it to be 0, and we define the first term to be `self[0]`.

INPUT:

- `alpha` – float; a smoothing factor with `0 <= alpha <= 1`

OUTPUT: a time series with the same number of steps as `self`

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.exponential_moving_average(0)
[0.0000, 1.0000, 1.0000, 1.0000, 1.0000]
sage: v.exponential_moving_average(1)
[0.0000, 1.0000, 1.0000, 1.0000, 2.0000]
sage: v.exponential_moving_average(0.5)
[0.0000, 1.0000, 1.0000, 1.0000, 1.5000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.exponential_moving_average(Integer(0))
[0.0000, 1.0000, 1.0000, 1.0000, 1.0000]
>>> v.exponential_moving_average(Integer(1))
[0.0000, 1.0000, 1.0000, 1.0000, 2.0000]
>>> v.exponential_moving_average(RealNumber('0.5'))
[0.0000, 1.0000, 1.0000, 1.0000, 1.5000]
```

Some more examples:

```
sage: v = stats.TimeSeries([1,2,3,4,5])
sage: v.exponential_moving_average(1)
[0.0000, 1.0000, 2.0000, 3.0000, 4.0000]
sage: v.exponential_moving_average(0)
[0.0000, 1.0000, 1.0000, 1.0000, 1.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(5)])
>>> v.exponential_moving_average(Integer(1))
[0.0000, 1.0000, 2.0000, 3.0000, 4.0000]
>>> v.exponential_moving_average(Integer(0))
[0.0000, 1.0000, 1.0000, 1.0000, 1.0000]
```

extend(*right*)

Extend this time series by appending elements from the iterable `right`.

INPUT:

- `right` – iterable that can be converted to a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1,2,-5]); v
[1.0000, 2.0000, -5.0000]
sage: v.extend([-3.5, 2])
sage: v
[1.0000, 2.0000, -5.0000, -3.5000, 2.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(2),-Integer(5)]); v
[1.0000, 2.0000, -5.0000]
>>> v.extend([-RealNumber('3.5'), Integer(2)])
>>> v
[1.0000, 2.0000, -5.0000, -3.5000, 2.0000]
```

fft(*overwrite=False*)

Return the real discrete fast Fourier transform of `self`, as a real time series:

$$[y(0), \Re(y(1)), \Im(y(1)), \ldots, \Re(y(n/2))] \text{ if } n \text{ is even}$$
$$[y(0), \Re(y(1)), \Im(y(1)), \ldots, \Re(y(n/2)), \Im(y(n/2))] \text{ if } n \text{ is odd}$$

where

$$y(j) = \sum_{k=0}^{n-1} x[k] \cdot \exp(-\sqrt{-1} \cdot jk \cdot 2\pi/n)$$

for $j = 0, \ldots, n-1$. Note that $y(-j) = y(n-j)$.

EXAMPLES:

```
sage: v = stats.TimeSeries([1..9]); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000]
sage: w = v.fft(); w
[45.0000, -4.5000, 12.3636, -4.5000, 5.3629, -4.5000, 2.5981, -4.5000, 0.7935]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(9)))); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000]
>>> w = v.fft(); w
[45.0000, -4.5000, 12.3636, -4.5000, 5.3629, -4.5000, 2.5981, -4.5000, 0.7935]
```

We get just the series of real parts of

```
sage: stats.TimeSeries([w[0]]) + w[1:].scale_time(2)
[45.0000, -4.5000, -4.5000, -4.5000, -4.5000]
```

```
>>> from sage.all import *
>>> stats.TimeSeries([w[Integer(0)]]) + w[Integer(1):].scale_time(Integer(2))
[45.0000, -4.5000, -4.5000, -4.5000, -4.5000]
```

An example with an even number of terms:

```
sage: v = stats.TimeSeries([1..10]); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
sage: w = v.fft(); w
[55.0000, -5.0000, 15.3884, -5.0000, 6.8819, -5.0000, 3.6327, -5.0000, 1.6246,
↪ -5.0000]
sage: v.fft().ifft()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10)))); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
>>> w = v.fft(); w
[55.0000, -5.0000, 15.3884, -5.0000, 6.8819, -5.0000, 3.6327, -5.0000, 1.6246,
↪ -5.0000]
>>> v.fft().ifft()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
```

histogram(*bins=50*, *normalize=False*)

Return the frequency histogram of the values in this time series divided up into the given number of bins.

INPUT:

- `bins` – positive integer (default: 50)
- `normalize` – boolean (default: `False`); whether to normalize so the total area in the bars of the histogram is 1

OUTPUT:

- counts – list of counts of numbers of elements in each bin
- endpoints – list of 2-tuples (a,b) that give the endpoints of the bins

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.histogram(3)
([1, 4, 3], [(-5.0, 0.0), (0.0, 5.0), (5.0, 10.0)])
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.histogram(Integer(3))
([1, 4, 3], [(-5.0, 0.0), (0.0, 5.0), (5.0, 10.0)])
```

hurst_exponent()

Return an estimate of the Hurst exponent of this time series.

We use the algorithm from pages 61 – 63 of [Peteres, Fractal Market Analysis (1994); see Google Books].

We define the Hurst exponent of a constant time series to be 1.

EXAMPLES:

The Hurst exponent of Brownian motion is 1/2. We approximate it with some specific samples. Note that the estimator is biased and slightly overestimates.

```
sage: set_random_seed(0)
sage: bm = stats.TimeSeries(10^5).randomize('normal').sums(); bm
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ...
 152.2437, 151.5327, 152.7629, 152.9169, 152.9084]
sage: bm.hurst_exponent()
0.527450972...
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> bm = stats.TimeSeries(Integer(10)**Integer(5)).randomize('normal').sums();
↪ bm
[0.6767, 0.2756, 0.6332, 0.0469, -0.8897 ...
 152.2437, 151.5327, 152.7629, 152.9169, 152.9084]
>>> bm.hurst_exponent()
0.527450972...
```

ifft(*overwrite=False*)

Return the real discrete inverse fast Fourier transform of `self`, which is also a real time series.

This is the inverse of `fft()`.

The returned real array contains

$$[y(0), y(1), \ldots, y(n-1)]$$

where for n is even

$$y(j) = 1/n \left(\sum_{k=1}^{n/2-1} (x[2k-1] + \sqrt{-1} \cdot x[2k]) \cdot \exp(\sqrt{-1} \cdot jk \cdot 2pi/n) + c.c. + x[0] + (-1)^j x[n-1] \right)$$

and for n is odd

$$y(j) = 1/n \left(\sum_{k=1}^{(n-1)/2} (x[2k-1] + \sqrt{-1} \cdot x[2k]) \cdot \exp(\sqrt{-1} \cdot jk \cdot 2pi/n) + c.c. + x[0] \right)$$

where $c.c.$ denotes complex conjugate of preceding expression.

EXAMPLES:

```
sage: v = stats.TimeSeries([1..10]); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
sage: v.ifft()
[5.1000, -5.6876, 1.4764, -1.0774, 0.4249, -0.1000, -0.2249, 0.6663, -1.2764,␣
↪1.6988]
sage: v.ifft().fft()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10)))); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
>>> v.ifft()
[5.1000, -5.6876, 1.4764, -1.0774, 0.4249, -0.1000, -0.2249, 0.6663, -1.2764,␣
↪1.6988]
>>> v.ifft().fft()
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000, 6.0000, 7.0000, 8.0000, 9.0000, 10.
↪0000]
```

list()

Return list of elements of `self`.

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-4,3,-2.5,-4,3])
sage: v.list()
[1.0, -4.0, 3.0, -2.5, -4.0, 3.0]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(4),Integer(3),-RealNumber('2.5
↪'),-Integer(4),Integer(3)])
>>> v.list()
[1.0, -4.0, 3.0, -2.5, -4.0, 3.0]
```

log()

Return new time series got by taking the logarithms of all the terms in the time series.

OUTPUT: a new time series

EXAMPLES:

We exponentiate then log a time series and get back the original series:

```
sage: v = stats.TimeSeries([1,-4,3,-2.5,-4,3]); v
[1.0000, -4.0000, 3.0000, -2.5000, -4.0000, 3.0000]
sage: v.exp()
[2.7183, 0.0183, 20.0855, 0.0821, 0.0183, 20.0855]
sage: v.exp().log()
[1.0000, -4.0000, 3.0000, -2.5000, -4.0000, 3.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(4),Integer(3),-RealNumber('2.5
↪'),-Integer(4),Integer(3)]); v
[1.0000, -4.0000, 3.0000, -2.5000, -4.0000, 3.0000]
>>> v.exp()
[2.7183, 0.0183, 20.0855, 0.0821, 0.0183, 20.0855]
>>> v.exp().log()
[1.0000, -4.0000, 3.0000, -2.5000, -4.0000, 3.0000]
```

Log of 0 gives `-inf`:

```
sage: stats.TimeSeries([1,0,3]).log()[1]
-inf
```

```
>>> from sage.all import *
>>> stats.TimeSeries([Integer(1),Integer(0),Integer(3)]).log()[Integer(1)]
-inf
```

max(*index=False*)

Return the largest value in this time series. If this series has length 0 we raise a `ValueError`.

INPUT:

- `index` – boolean (default: `False`); if `True`, also return index of maximum entry

OUTPUT:

- float; largest value
- integer; index of largest value (only returned if `index=True`)

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-4,3,-2.5,-4,3])
sage: v.max()
3.0
sage: v.max(index=True)
(3.0, 2)
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(4),Integer(3),-RealNumber('2.5
↪'),-Integer(4),Integer(3)])
>>> v.max()
3.0
>>> v.max(index=True)
(3.0, 2)
```

mean()

Return the mean (average) of the elements of `self`.

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.mean()
1.6
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.mean()
1.6
```

min(*index=False*)

Return the smallest value in this time series.

If this series has length 0, we raise a `ValueError`.

INPUT:

- `index` – boolean (default: `False`); if `True`, also return index of minimal entry

OUTPUT:

- float; smallest value
- integer; index of smallest value (only returned if `index=True`)

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-4,3,-2.5,-4])
sage: v.min()
-4.0
sage: v.min(index=True)
(-4.0, 1)
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(4),Integer(3),-RealNumber('2.5
↪'),-Integer(4)])
>>> v.min()
-4.0
>>> v.min(index=True)
(-4.0, 1)
```

moment(k)

Return the k-th moment of `self`, which is just the mean of the k-th powers of the elements of `self`.

INPUT:

- `k` – positive integer

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.moment(1)
1.6
sage: v.moment(2)
3.2
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.moment(Integer(1))
1.6
>>> v.moment(Integer(2))
3.2
```

numpy(*copy=True*)

Return a NumPy version of this time series.

Note

If copy is `False`, return a NumPy 1-D array reference to exactly the same block of memory as this time series. This is very, very fast and makes it easy to quickly use all NumPy/SciPy functionality on `self`. However, it is dangerous because when this time series goes out of scope and is garbage collected, the corresponding NumPy reference object will point to garbage.

INPUT:

- `copy` – boolean (default: `True`)

OUTPUT: a numpy 1-D array

EXAMPLES:

```
sage: v = stats.TimeSeries([1,-3,4.5,-2])
sage: w = v.numpy(copy=False); w
array([ 1. , -3. ,  4.5, -2. ])
sage: type(w)
<... 'numpy.ndarray'>
sage: w.shape
(4,)
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),-Integer(3),RealNumber('4.5'),-
↪Integer(2)])
>>> w = v.numpy(copy=False); w
array([ 1. , -3. ,  4.5, -2. ])
>>> type(w)
<... 'numpy.ndarray'>
>>> w.shape
(4,)
```

Notice that changing `w` also changes `v` too!

```
sage: w[0] = 20
sage: w
array([20. , -3. ,  4.5, -2. ])
sage: v
[20.0000, -3.0000, 4.5000, -2.0000]
```

```
>>> from sage.all import *
>>> w[Integer(0)] = Integer(20)
>>> w
array([20. , -3. ,  4.5, -2. ])
>>> v
[20.0000, -3.0000, 4.5000, -2.0000]
```

If you want a separate copy do not give the `copy=False` option.

```
sage: z = v.numpy(); z
array([20. , -3. ,  4.5, -2. ])
sage: z[0] = -10
sage: v
[20.0000, -3.0000, 4.5000, -2.0000]
```

```
>>> from sage.all import *
>>> z = v.numpy(); z
array([20. , -3. ,  4.5, -2. ])
>>> z[Integer(0)] = -Integer(10)
>>> v
[20.0000, -3.0000, 4.5000, -2.0000]
```

plot(*plot_points=1000*, *points=False*, ***kwds*)

Return a plot of this time series.

The plot shows a line or points through $(i, T(i))$, where i ranges over nonnegative integers up to the length of `self`.

INPUT:

- `plot_points` – (default: 1000) 0 or positive integer. Only plot the given number of equally spaced points in the time series. If 0, plot all points.
- `points` – boolean (default: `False`); if `True`, return just the points of the time series
- `**kwds` – passed to the line or point command

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.plot()
Graphics object consisting of 1 graphics primitive
sage: v.plot(points=True)
Graphics object consisting of 1 graphics primitive
sage: v.plot() + v.plot(points=True, rgbcolor='red')
Graphics object consisting of 2 graphics primitives
sage: v.plot() + v.plot(points=True, rgbcolor='red', pointsize=50)
Graphics object consisting of 2 graphics primitives
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.plot()
Graphics object consisting of 1 graphics primitive
>>> v.plot(points=True)
Graphics object consisting of 1 graphics primitive
>>> v.plot() + v.plot(points=True, rgbcolor='red')
Graphics object consisting of 2 graphics primitives
>>> v.plot() + v.plot(points=True, rgbcolor='red', pointsize=Integer(50))
Graphics object consisting of 2 graphics primitives
```

plot_candlestick(*bins=30*)

Return a candlestick plot of this time series with the given number of bins.

A candlestick plot is a style of bar-chart used to view open, high, low, and close stock data. At each bin, the line represents the high / low range. The bar represents the open / close range. The interval is colored blue if the open for that bin is less than the close. If the close is less than the open, then that bin is colored red instead.

INPUT:

- `bins` – positive integer (default: 30); the number of bins or candles

OUTPUT: a candlestick plot

EXAMPLES:

Here we look at the candlestick plot for Brownian motion:

```
sage: v = stats.TimeSeries(1000).randomize()
sage: v.plot_candlestick(bins=20)
Graphics object consisting of 40 graphics primitives
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries(Integer(1000)).randomize()
>>> v.plot_candlestick(bins=Integer(20))
Graphics object consisting of 40 graphics primitives
```

plot_histogram(*bins=50*, *normalize=True*, ***kwds*)

Return histogram plot of this time series with given number of bins.

INPUT:

- `bins` – positive integer (default: 50)
- `normalize` – boolean (default: `True`); whether to normalize so the total area in the bars of the histogram is 1

OUTPUT: a histogram plot

EXAMPLES:

```
sage: v = stats.TimeSeries([1..50])
sage: v.plot_histogram(bins=10)
Graphics object consisting of 10 graphics primitives
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(50))))
>>> v.plot_histogram(bins=Integer(10))
Graphics object consisting of 10 graphics primitives
```

```
sage: v.plot_histogram(bins=3,normalize=False,aspect_ratio=1)
Graphics object consisting of 3 graphics primitives
```

```
>>> from sage.all import *
>>> v.plot_histogram(bins=Integer(3),normalize=False,aspect_ratio=Integer(1))
Graphics object consisting of 3 graphics primitives
```

pow(*k*)

Return a new time series with all elements of `self` raised to the k-th power.

INPUT:

- `k` – a float

OUTPUT: a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.pow(2)
[1.0000, 1.0000, 1.0000, 4.0000, 9.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.pow(Integer(2))
[1.0000, 1.0000, 1.0000, 4.0000, 9.0000]
```

prod()

Return the product of all the entries of `self`.

If `self` has length 0, returns 1.

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.prod()
6.0
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.prod()
6.0
```

randomize(*distribution='uniform'*, *loc=0*, *scale=1*, ***kwds*)

Randomize the entries in this time series, and return a reference to `self`. Thus this function both changes `self` in place, and returns a copy of `self`, for convenience.

INPUT:

- `distribution` – (default: `'uniform'`) supported values are:
  - `'uniform'` – from `loc` to `loc + scale`
  - `'normal'` – mean `loc` and standard deviation `scale`
  - `'semicircle'` – with center at `loc` (`scale` is ignored)
  - `'lognormal'` – mean `loc` and standard deviation `scale`
- `loc` – float (default: 0)
- `scale` – float (default: 1)

Note

All random numbers are generated using algorithms that build on the high quality GMP random number function gmp_urandomb_ui. Thus this function fully respects the Sage `set_random_state` command. It's not quite as fast as the C library random number generator, but is of course much better quality, and is platform independent.

EXAMPLES:

We generate 5 uniform random numbers in the interval [0,1]:

```
sage: set_random_seed(0)
sage: stats.TimeSeries(5).randomize()
[0.8685, 0.2816, 0.0229, 0.1456, 0.7314]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> stats.TimeSeries(Integer(5)).randomize()
[0.8685, 0.2816, 0.0229, 0.1456, 0.7314]
```

We generate 5 uniform random numbers from 5 to $5 + 2 = 7$:

```
sage: set_random_seed(0)
sage: stats.TimeSeries(10).randomize('uniform', 5, 2)
[6.7369, 5.5632, 5.0459, 5.2911, 6.4628, 5.2412, 5.2010, 5.2761, 5.5813, 5.
↪5439]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> stats.TimeSeries(Integer(10)).randomize('uniform', Integer(5), Integer(2))
[6.7369, 5.5632, 5.0459, 5.2911, 6.4628, 5.2412, 5.2010, 5.2761, 5.5813, 5.
↪5439]
```

We generate 5 normal random values with mean 0 and variance 1.

```
sage: set_random_seed(0)
sage: stats.TimeSeries(5).randomize('normal')
[0.6767, -0.4011, 0.3576, -0.5864, -0.9365]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> stats.TimeSeries(Integer(5)).randomize('normal')
[0.6767, -0.4011, 0.3576, -0.5864, -0.9365]
```

We generate 10 normal random values with mean 5 and variance 2.

```
sage: set_random_seed(0)
sage: stats.TimeSeries(10).randomize('normal', 5, 2)
[6.3534, 4.1978, 5.7153, 3.8273, 3.1269, 2.9598, 3.7484, 6.7472, 3.8986, 4.
↪6271]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> stats.TimeSeries(Integer(10)).randomize('normal', Integer(5), Integer(2))
[6.3534, 4.1978, 5.7153, 3.8273, 3.1269, 2.9598, 3.7484, 6.7472, 3.8986, 4.
↪6271]
```

We generate 5 values using the semicircle distribution.

```
sage: set_random_seed(0)
sage: stats.TimeSeries(5).randomize('semicircle')
[0.7369, -0.9541, 0.4628, -0.7990, -0.4187]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> stats.TimeSeries(Integer(5)).randomize('semicircle')
[0.7369, -0.9541, 0.4628, -0.7990, -0.4187]
```

We generate 1 million normal random values and create a frequency histogram.

```
sage: set_random_seed(0)
sage: a = stats.TimeSeries(10^6).randomize('normal')
sage: a.histogram(10)[0]
[36, 1148, 19604, 130699, 340054, 347870, 137953, 21290, 1311, 35]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> a = stats.TimeSeries(Integer(10)**Integer(6)).randomize('normal')
>>> a.histogram(Integer(10))[Integer(0)]
[36, 1148, 19604, 130699, 340054, 347870, 137953, 21290, 1311, 35]
```

We take the above values, and compute the proportion that lie within 1, 2, 3, 5, and 6 standard deviations of the mean (0):

```
sage: s = a.standard_deviation()
sage: len(a.clip_remove(-s,s))/float(len(a))
0.683094
```

(continues on next page)

(continued from previous page)

```
sage: len(a.clip_remove(-2*s,2*s))/float(len(a))
0.954559
sage: len(a.clip_remove(-3*s,3*s))/float(len(a))
0.997228
sage: len(a.clip_remove(-5*s,5*s))/float(len(a))
0.999998
```

```
>>> from sage.all import *
>>> s = a.standard_deviation()
>>> len(a.clip_remove(-s,s))/float(len(a))
0.683094
>>> len(a.clip_remove(-Integer(2)*s,Integer(2)*s))/float(len(a))
0.954559
>>> len(a.clip_remove(-Integer(3)*s,Integer(3)*s))/float(len(a))
0.997228
>>> len(a.clip_remove(-Integer(5)*s,Integer(5)*s))/float(len(a))
0.999998
```

There were no "six sigma events":

```
sage: len(a.clip_remove(-6*s,6*s))/float(len(a))
1.0
```

```
>>> from sage.all import *
>>> len(a.clip_remove(-Integer(6)*s,Integer(6)*s))/float(len(a))
1.0
```

range_statistic(*b=None*)

Return the rescaled range statistic R/S of `self`, which is defined as follows (see Hurst 1951). If the optional parameter `b` is given, return the average of R/S range statistics of disjoint blocks of size `b`.

Let σ be the standard deviation of the sequence of differences of `self`, and let Y_k be the k-th term of `self`. Let n be the number of terms of `self`, and set $Z_k = Y_k - ((k+1)/n) \cdot Y_n$. Then

$$R/S = \big(\max(Z_k) - \min(Z_k)\big)/\sigma$$

where the max and min are over all Z_k. Basically replacing Y_k by Z_k allows us to measure the difference from the line from the origin to (n, Y_n).

INPUT:

- `self` – a time series (*not* the series of differences)
- `b` – integer (default: `None`); if given instead divide the input time series up into `j = floor(n/b)` disjoint blocks, compute the R/S statistic for each block, and return the average of those R/S statistics.

OUTPUT: float

EXAMPLES:

Notice that if we make a Brownian motion random walk, there is no difference if we change the standard deviation.

```
sage: set_random_seed(0); stats.TimeSeries(10^6).randomize('normal').sums().
↪range_statistic()
```

(continues on next page)

(continued from previous page)

```
1897.8392602...
sage: set_random_seed(0); stats.TimeSeries(10^6).randomize('normal',0,100).
↪sums().range_statistic()
1897.8392602...
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0)); stats.TimeSeries(Integer(10)**Integer(6)).
↪randomize('normal').sums().range_statistic()
1897.8392602...
>>> set_random_seed(Integer(0)); stats.TimeSeries(Integer(10)**Integer(6)).
↪randomize('normal',Integer(0),Integer(100)).sums().range_statistic()
1897.8392602...
```

rescale(*s*)

Change `self` by multiplying every value in the series by `s`.

INPUT:

- `s` – a float

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.rescale(0.5)
sage: v
[2.5000, 2.0000, 0.6500, 1.0000, 4.0000, 5.0000, 1.5000, -2.5000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.rescale(RealNumber('0.5'))
>>> v
[2.5000, 2.0000, 0.6500, 1.0000, 4.0000, 5.0000, 1.5000, -2.5000]
```

reversed()

Return new time series obtain from this time series by reversing the order of the entries in this time series.

OUTPUT: a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1..5])
sage: v.reversed()
[5.0000, 4.0000, 3.0000, 2.0000, 1.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(5))))
>>> v.reversed()
[5.0000, 4.0000, 3.0000, 2.0000, 1.0000]
```

scale(*s*)

Return new time series obtained by multiplying every value in the series by `s`.

INPUT:

- s – a float

OUTPUT: a new time series with all values multiplied by s

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.scale(0.5)
[2.5000, 2.0000, 0.6500, 1.0000, 4.0000, 5.0000, 1.5000, -2.5000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.scale(RealNumber('0.5'))
[2.5000, 2.0000, 0.6500, 1.0000, 4.0000, 5.0000, 1.5000, -2.5000]
```

scale_time(k)

Return the new time series at scale k. If the input time series is $X_0, X_1, X_2, \ldots$, then this function returns the shorter time series $X_0, X_k, X_{2k}, \ldots$.

INPUT:

- k – positive integer

OUTPUT: a new time series

EXAMPLES:

```
sage: v = stats.TimeSeries([5,4,1.3,2,8,10,3,-5]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.scale_time(1)
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
sage: v.scale_time(2)
[5.0000, 1.3000, 8.0000, 3.0000]
sage: v.scale_time(3)
[5.0000, 2.0000]
sage: v.scale_time(10)
[]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(5),Integer(4),RealNumber('1.3'),Integer(2),
↪Integer(8),Integer(10),Integer(3),-Integer(5)]); v
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.scale_time(Integer(1))
[5.0000, 4.0000, 1.3000, 2.0000, 8.0000, 10.0000, 3.0000, -5.0000]
>>> v.scale_time(Integer(2))
[5.0000, 1.3000, 8.0000, 3.0000]
>>> v.scale_time(Integer(3))
[5.0000, 2.0000]
>>> v.scale_time(Integer(10))
[]
```

A series of odd length:

```
sage: v = stats.TimeSeries([1..5]); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
sage: v.scale_time(2)
[1.0000, 3.0000, 5.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(5)))); v
[1.0000, 2.0000, 3.0000, 4.0000, 5.0000]
>>> v.scale_time(Integer(2))
[1.0000, 3.0000, 5.0000]
```

show(**args*, ***kwds*)

Return a plot of this time series.

This is an alias of *plot()*.

EXAMPLES:

Draw a plot of a time series:

```
sage: stats.TimeSeries([1..10]).show()
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10)))).show()
Graphics object consisting of 1 graphics primitive
```

simple_moving_average(k)

Return the moving average time series over the last k time units.

Assumes the input time series was constant with its starting value for negative time. The t-th step of the output is the sum of the previous $k-1$ steps of `self` and the k-th step divided by k. Thus k values are averaged at each point.

INPUT:

- `k` – positive integer

OUTPUT: a time series with the same number of steps as `self`

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.simple_moving_average(0)
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.simple_moving_average(1)
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.simple_moving_average(2)
[1.0000, 1.0000, 1.0000, 1.5000, 2.5000]
sage: v.simple_moving_average(3)
[1.0000, 1.0000, 1.0000, 1.3333, 2.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
```

(continues on next page)

(continued from previous page)

```
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.simple_moving_average(Integer(0))
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.simple_moving_average(Integer(1))
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.simple_moving_average(Integer(2))
[1.0000, 1.0000, 1.0000, 1.5000, 2.5000]
>>> v.simple_moving_average(Integer(3))
[1.0000, 1.0000, 1.0000, 1.3333, 2.0000]
```

standard_deviation(*bias=False*)

Return the standard deviation of the entries of `self`.

INPUT:

- `bias` – boolean (default: `False`); if `False`, divide by `self.length() - 1` instead of `self.length()` to give a less biased estimator for the variance.

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.standard_deviation()
0.8944271909...
sage: v.standard_deviation(bias=True)
0.8
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.standard_deviation()
0.8944271909...
>>> v.standard_deviation(bias=True)
0.8
```

sum()

Return the sum of all the entries of `self`.

If `self` has length 0, returns 0.

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.sum()
8.0
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
```

(continues on next page)

(continued from previous page)

```
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.sum()
8.0
```

sums(*s=0*)

Return the new time series got by taking the running partial sums of the terms of this time series.

INPUT:

- `s` – starting value for partial sums

OUTPUT: a time series

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.sums()
[1.0000, 2.0000, 3.0000, 5.0000, 8.0000]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.sums()
[1.0000, 2.0000, 3.0000, 5.0000, 8.0000]
```

variance(*bias=False*)

Return the variance of the elements of `self`, which is the mean of the squares of the differences from the mean.

INPUT:

- `bias` – boolean (default: `False`); if `False`, divide by `self.length() - 1` instead of `self.length()` to give a less biased estimator for the variance.

OUTPUT: double

EXAMPLES:

```
sage: v = stats.TimeSeries([1,1,1,2,3]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
sage: v.variance()
0.8
sage: v.variance(bias=True)
0.64
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(3)]); v
[1.0000, 1.0000, 1.0000, 2.0000, 3.0000]
>>> v.variance()
0.8
>>> v.variance(bias=True)
0.64
```

vector()

Return real double vector whose entries are the values of this time series. This is useful since vectors have standard algebraic structure and play well with matrices.

OUTPUT: a real double vector

EXAMPLES:

```
sage: v = stats.TimeSeries([1..10])
sage: v.vector()
(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0)
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries((ellipsis_range(Integer(1),Ellipsis,Integer(10))))
>>> v.vector()
(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0)
```

sage.stats.time_series.**autoregressive_fit**(*acvs*)

Given a sequence of lagged autocovariances of length M produce $a_1, \ldots, a_p$ so that the first M autocovariance coefficients of the autoregressive processes $X_t = a_1 X_{t_1} + \cdots + a_p X_{t-p} + Z_t$ are the same as the input sequence.

The function works by solving the Yule-Walker equations $\Gamma a = \gamma$, where $\gamma = (\gamma(1), \ldots, \gamma(M))$, $a = (a_1, \ldots, a_M)$, with $\gamma(i)$ the autocovariance of lag i and $\Gamma_{ij} = \gamma(i-j)$.

sage.stats.time_series.**unpickle_time_series_v1**(*v*, *n*)

Version 1 unpickle method.

INPUT:

- v – a raw char buffer

EXAMPLES:

```
sage: v = stats.TimeSeries([1,2,3])
sage: s = v.__reduce__()[1][0]
sage: type(s)
<class 'bytes'>
sage: sage.stats.time_series.unpickle_time_series_v1(s,3)
[1.0000, 2.0000, 3.0000]
sage: sage.stats.time_series.unpickle_time_series_v1(s+s,6)
[1.0000, 2.0000, 3.0000, 1.0000, 2.0000, 3.0000]
sage: sage.stats.time_series.unpickle_time_series_v1(b'',0)
[]
```

```
>>> from sage.all import *
>>> v = stats.TimeSeries([Integer(1),Integer(2),Integer(3)])
>>> s = v.__reduce__()[Integer(1)][Integer(0)]
>>> type(s)
<class 'bytes'>
>>> sage.stats.time_series.unpickle_time_series_v1(s,Integer(3))
[1.0000, 2.0000, 3.0000]
>>> sage.stats.time_series.unpickle_time_series_v1(s+s,Integer(6))
[1.0000, 2.0000, 3.0000, 1.0000, 2.0000, 3.0000]
>>> sage.stats.time_series.unpickle_time_series_v1(b'',Integer(0))
[]
```

CHAPTER

FOUR

HIDDEN MARKOV MODELS

This is a complete pure-Cython optimized implementation of Hidden Markov Models. It fully supports Discrete, Gaussian, and Mixed Gaussian emissions.

The best references for the basic HMM algorithms implemented here are:

- Tapas Kanungo's "Hidden Markov Models"
- **Jackson's HMM tutorial:**
  http://personal.ee.surrey.ac.uk/Personal/P.Jackson/tutorial/

LICENSE: Some of the code in this file is based on reading Kanungo's GPLv2+ implementation of discrete HMM's, hence the present code must be licensed with a GPLv2+ compatible license.

AUTHOR:

- William Stein, 2010-03

class sage.stats.hmm.hmm.**DiscreteHiddenMarkovModel**

Bases: *HiddenMarkovModel*

A discrete Hidden Markov model implemented using double precision floating point arithmetic.

INPUT:

- `A` – list of lists or a square $N \times N$ matrix, whose (i, j) entry gives the probability of transitioning from state i to state j.
- `B` – list of N lists or a matrix with N rows, such that $B[i, k]$ gives the probability of emitting symbol k while in state i.
- `pi` – the probabilities of starting in each initial state, i.e., `pi[i]` is the probability of starting in state i.
- `emission_symbols` – `None` or list (default: `None`); if None, the emission_symbols are the ints `[0..N-1]`, where N is the number of states. Otherwise, they are the entries of the list `emissions_symbols`, which must all be hashable.
- `normalize` – boolean (default: `True`); if given, input is normalized to define valid probability distributions, e.g., the entries of A are made nonnegative and the rows sum to 1, and the probabilities in `pi` are normalized.

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.5,.5]); m
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[0.4 0.6]
[0.1 0.9]
```

(continues on next page)

(continued from previous page)

```
Emission matrix:
[0.1 0.9]
[0.5 0.5]
Initial probabilities: [0.5000, 0.5000]
sage: m.log_likelihood([0,1,0,1,0,1])
-4.66693474691329...
sage: m.viterbi([0,1,0,1,0,1])
([1, 1, 1, 1, 1, 1], -5.378832842208748)
sage: m.baum_welch([0,1,0,1,0,1])
(0.0, 22)
sage: m  # rel tol 1e-10
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[1.0134345614745788e-70                    1.0]
[                   1.0 3.9974352713558623e-19]
Emission matrix:
[ 7.380221566254936e-54                    1.0]
[                   1.0 3.9974352626002193e-19]
Initial probabilities: [0.0000, 1.0000]
sage: m.sample(10)
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
sage: m.graph().plot()
Graphics object consisting of 6 graphics primitives
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.5'),RealNumber('.5')]); m
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[0.4 0.6]
[0.1 0.9]
Emission matrix:
[0.1 0.9]
[0.5 0.5]
Initial probabilities: [0.5000, 0.5000]
>>> m.log_likelihood([Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(1)])
-4.66693474691329...
>>> m.viterbi([Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),Integer(1)])
([1, 1, 1, 1, 1, 1], -5.378832842208748)
>>> m.baum_welch([Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(1)])
(0.0, 22)
>>> m  # rel tol 1e-10
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[1.0134345614745788e-70                    1.0]
[                   1.0 3.9974352713558623e-19]
Emission matrix:
```

(continues on next page)

(continued from previous page)

```
[ 7.380221566254936e-54                    1.0]
[                   1.0 3.9974352626002193e-19]
Initial probabilities: [0.0000, 1.0000]
>>> m.sample(Integer(10))
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
>>> m.graph().plot()
Graphics object consisting of 6 graphics primitives
```

A 3-state model that happens to always outputs 'b':

```
sage: m = hmm.DiscreteHiddenMarkovModel([[1/3]*3]*3, [[0,1,0]]*3, [1/3]*3, ['a','b
↪','c'])
sage: m.sample(10)
['b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[Integer(1)/
↪Integer(3)]*Integer(3)]*Integer(3), [[Integer(0),Integer(1),
↪Integer(0)]]*Integer(3), [Integer(1)/Integer(3)]*Integer(3), ['a','b','c'])
>>> m.sample(Integer(10))
['b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']
```

baum_welch(*obs*, *max_iter=100*, *log_likelihood_cutoff=0.0001*, *fix_emissions=False*)

Given an observation sequence obs, improve this HMM using the Baum-Welch algorithm to increase the probability of observing obs.

INPUT:

- `obs` – list of emissions
- `max_iter` – integer (default: 100); maximum number of Baum-Welch steps to take
- `log_likelihood_cutoff` – positive float (default: 1e-4); the minimal improvement in likelihood with respect to the last iteration required to continue. Relative value to log likelihood.
- `fix_emissions` – boolean (default: `False`); if `True`, do not change emissions when updating

OUTPUT:

changes the model in place, and returns the log likelihood and number of iterations.

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.9,0.1]],
....:                                   [[.5,.5],[0,1]],
....:                                   [.2,.8])
sage: m.baum_welch([1,0]*20, log_likelihood_cutoff=0)
(0.0, 4)
sage: m  # rel tol 1e-14
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[1.3515269707707603e-51                    1.0]
[                   1.0                    0.0]
Emission matrix:
[                  1.0 6.462537138850569e-52]
```

(continues on next page)

(continued from previous page)

```
[                  0.0                   1.0]
Initial probabilities: [0.0000, 1.0000]
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.9'),RealNumber('0.1')]],
...                                   [[RealNumber('.5'),RealNumber('.5')],
↪[Integer(0),Integer(1)]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> m.baum_welch([Integer(1),Integer(0)]*Integer(20), log_likelihood_
↪cutoff=Integer(0))
(0.0, 4)
>>> m  # rel tol 1e-14
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[1.3515269707707603e-51                    1.0]
[                   1.0                    0.0]
Emission matrix:
[                  1.0 6.462537138850569e-52]
[                  0.0                   1.0]
Initial probabilities: [0.0000, 1.0000]
```

The following illustrates how Baum-Welch is only a local optimizer, i.e., the above model is far more likely to produce the sequence [1,0]*20 than the one we get below:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.5,0.5],[0.5,0.5]],
....:                                   [[.5,.5],[.5,.5]],
....:                                   [.5,.5])
sage: m.baum_welch([1,0]*20, log_likelihood_cutoff=0)
(-27.725887222397784, 1)
sage: m
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
[0.5 0.5]
[0.5 0.5]
Emission matrix:
[0.5 0.5]
[0.5 0.5]
Initial probabilities: [0.5000, 0.5000]
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.5'),RealNumber('0.5')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [[RealNumber('.5'),RealNumber('.5')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [RealNumber('.5'),RealNumber('.5')])
>>> m.baum_welch([Integer(1),Integer(0)]*Integer(20), log_likelihood_
↪cutoff=Integer(0))
(-27.725887222397784, 1)
>>> m
Discrete Hidden Markov Model with 2 States and 2 Emissions
Transition matrix:
```

(continues on next page)

(continued from previous page)

```
[0.5 0.5]
[0.5 0.5]
Emission matrix:
[0.5 0.5]
[0.5 0.5]
Initial probabilities: [0.5000, 0.5000]
```

We illustrate fixing emissions:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.9,0.1]],
....:                                   [[.5,.5],[.2,.8]],
....:                                   [.2,.8])
sage: set_random_seed(0); v = m.sample(100)
sage: m.baum_welch(v,fix_emissions=True)
(-66.98630856918774, 100)
sage: m.emission_matrix()
[0.5 0.5]
[0.2 0.8]
sage: m = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.9,0.1]],
....:                                   [[.5,.5],[.2,.8]],
....:                                   [.2,.8])
sage: m.baum_welch(v)
(-66.782360659293..., 100)
sage: m.emission_matrix()  # rel tol 1e-14
[ 0.5303085748626447 0.46969142513735535]
[ 0.2909775550173978  0.7090224449826023]
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.9'),RealNumber('0.1')]],
...                                   [[RealNumber('.5'),RealNumber('.5')],
↪[RealNumber('.2'),RealNumber('.8')]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> set_random_seed(Integer(0)); v = m.sample(Integer(100))
>>> m.baum_welch(v,fix_emissions=True)
(-66.98630856918774, 100)
>>> m.emission_matrix()
[0.5 0.5]
[0.2 0.8]
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.9'),RealNumber('0.1')]],
...                                   [[RealNumber('.5'),RealNumber('.5')],
↪[RealNumber('.2'),RealNumber('.8')]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> m.baum_welch(v)
(-66.782360659293..., 100)
>>> m.emission_matrix()  # rel tol 1e-14
[ 0.5303085748626447 0.46969142513735535]
[ 0.2909775550173978  0.7090224449826023]
```

emission_matrix()

Return the matrix whose i-th row specifies the emission probability distribution for the i-th state.

More precisely, the i, j entry of the matrix is the probability of the Markov model outputting the j-th symbol when it is in the i-th state.

OUTPUT: a Sage matrix with real double precision (RDF) entries.

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.5,.5])
sage: E = m.emission_matrix(); E
[0.1 0.9]
[0.5 0.5]
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.5'),RealNumber('.5')])
>>> E = m.emission_matrix(); E
[0.1 0.9]
[0.5 0.5]
```

The returned matrix is mutable, but changing it does not change the transition matrix for the model:

```
sage: E[0,0] = 0; E[0,1] = 1
sage: m.emission_matrix()
[0.1 0.9]
[0.5 0.5]
```

```
>>> from sage.all import *
>>> E[Integer(0),Integer(0)] = Integer(0); E[Integer(0),Integer(1)] =␣
↪Integer(1)
>>> m.emission_matrix()
[0.1 0.9]
[0.5 0.5]
```

generate_sequence(*length*, *starting_state=None*)

Return a sample of the given length from this HMM.

INPUT:

- `length` – positive integer
- `starting_state` – integer (or `None`); if specified, generate a sequence using this model starting with the given state instead of the initial probabilities to determine the starting state

OUTPUT:

- an *IntList* or list of emission symbols
- *IntList* of the actual states the model was in when emitting the corresponding symbols

EXAMPLES:

In this example, the emission symbols are not set:

```
sage: set_random_seed(0)
sage: a = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.1,0.9]],
....:                                   [[1,0],[0,1]],
....:                                   [0,1])
sage: a.generate_sequence(5)
([1, 0, 1, 1, 1], [1, 0, 1, 1, 1])
sage: list(a.generate_sequence(1000)[0]).count(0)
90
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[Integer(1),Integer(0)],[Integer(0),
↪Integer(1)]],
...                                   [Integer(0),Integer(1)])
>>> a.generate_sequence(Integer(5))
([1, 0, 1, 1, 1], [1, 0, 1, 1, 1])
>>> list(a.generate_sequence(Integer(1000))[Integer(0)]).count(Integer(0))
90
```

Here the emission symbols are set:

```
sage: set_random_seed(0)
sage: a = hmm.DiscreteHiddenMarkovModel([[0.5,0.5],[0.1,0.9]],
....:                                   [[1,0],[0,1]],
....:                                   [0,1], ['up', 'down'])
sage: a.generate_sequence(5)
(['down', 'up', 'down', 'down', 'down'], [1, 0, 1, 1, 1])
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.5'),RealNumber('0.5')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[Integer(1),Integer(0)],[Integer(0),
↪Integer(1)]],
...                                   [Integer(0),Integer(1)], ['up', 'down'])
>>> a.generate_sequence(Integer(5))
(['down', 'up', 'down', 'down', 'down'], [1, 0, 1, 1, 1])
```

Specify the starting state:

```
sage: set_random_seed(0); a.generate_sequence(5, starting_state=0)
(['up', 'up', 'down', 'down', 'down'], [0, 0, 1, 1, 1])
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0)); a.generate_sequence(Integer(5), starting_
↪state=Integer(0))
(['up', 'up', 'down', 'down', 'down'], [0, 0, 1, 1, 1])
```

log_likelihood(*obs*, *scale=True*)

Return the logarithm of the probability that this model produced the given observation sequence. Thus the output is a nonpositive number.

INPUT:

- obs – sequence of observations
- scale – boolean (default: True); if True, use rescaling to overoid loss of precision due to the very limited dynamic range of floats. You should leave this as True unless the obs sequence is very small.

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.2,.8])
sage: m.log_likelihood([0, 1, 0, 1, 1, 0, 1, 0, 0, 0])
-7.3301308009370825
sage: m.log_likelihood([0, 1, 0, 1, 1, 0, 1, 0, 0, 0], scale=False)
-7.330130800937082
sage: m.log_likelihood([])
0.0

sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.2,.8], ['happy','sad'])
sage: m.log_likelihood(['happy','happy'])
-1.6565295199679506
sage: m.log_likelihood(['happy','sad'])
-1.4731602941415523
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> m.log_likelihood([Integer(0), Integer(1), Integer(0), Integer(1),␣
↪Integer(1), Integer(0), Integer(1), Integer(0), Integer(0), Integer(0)])
-7.3301308009370825
>>> m.log_likelihood([Integer(0), Integer(1), Integer(0), Integer(1),␣
↪Integer(1), Integer(0), Integer(1), Integer(0), Integer(0), Integer(0)],␣
↪scale=False)
-7.330130800937082
>>> m.log_likelihood([])
0.0

>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.2'),RealNumber('.8')], [
↪'happy','sad'])
>>> m.log_likelihood(['happy','happy'])
-1.6565295199679506
>>> m.log_likelihood(['happy','sad'])
-1.4731602941415523
```

Overflow from not using the scale option:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.2,.8])
sage: m.log_likelihood([0,1]*1000, scale=True)
-1433.820666652728
sage: m.log_likelihood([0,1]*1000, scale=False)
-inf
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> m.log_likelihood([Integer(0),Integer(1)]*Integer(1000), scale=True)
-1433.820666652728
>>> m.log_likelihood([Integer(0),Integer(1)]*Integer(1000), scale=False)
-inf
```

viterbi(*obs*, *log_scale=True*)

Determine "the" hidden sequence of states that is most likely to produce the given sequence seq of observations, along with the probability that this hidden sequence actually produced the observation.

INPUT:

- `seq` – sequence of emitted ints or symbols
- `log_scale` – boolean (default: `True`); whether to scale the sequence in order to avoid numerical overflow

OUTPUT:

- `list` – "the" most probable sequence of hidden states, i.e., the Viterbi path
- `float` – log of probability that the observed sequence was produced by the Viterbi sequence of states

EXAMPLES:

```
sage: a = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.1,0.9]],
....:                                   [[0.9,0.1],[0.1,0.9]],
....:                                   [0.5,0.5])
sage: a.viterbi([1,0,0,1,0,0,1,1])
([1, 0, 0, 1, ..., 0, 1, 1], -11.06245322477221...)
```

```
>>> from sage.all import *
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.9'),RealNumber('0.1')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [RealNumber('0.5'),RealNumber('0.5')])
>>> a.viterbi([Integer(1),Integer(0),Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(1),Integer(1)])
([1, 0, 0, 1, ..., 0, 1, 1], -11.06245322477221...)
```

We predict the state sequence when the emissions are 3/4 and 'abc'.:

```
sage: a = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.1,0.9]],
....:                                   [[0.9,0.1],[0.1,0.9]],
....:                                   [0.5,0.5], [3/4, 'abc'])
```

```
>>> from sage.all import *
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.9'),RealNumber('0.1')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [RealNumber('0.5'),RealNumber('0.5')],␣
↪[Integer(3)/Integer(4), 'abc'])
```

Note that state 0 is common below, despite the model trying hard to switch to state 1:

```
sage: a.viterbi([3/4, 'abc', 'abc'] + [3/4]*10)
([0, 1, 1, 0, 0 ... 0, 0, 0, 0, 0], -25.299405845367794)
```

```
>>> from sage.all import *
>>> a.viterbi([Integer(3)/Integer(4), 'abc', 'abc'] + [Integer(3)/
↪Integer(4)]*Integer(10))
([0, 1, 1, 0, 0 ... 0, 0, 0, 0, 0], -25.299405845367794)
```

class sage.stats.hmm.hmm.**HiddenMarkovModel**

Bases: object

Abstract base class for all Hidden Markov Models.

graph(*eps=0.001*)

Create a weighted directed graph from the transition matrix, not including any edge with a probability less than eps.

INPUT:

- eps – nonnegative real number

OUTPUT: a DiGraph

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[.3,0,.7],[0,0,1],[.5,.5,0]],
....:                                   [[.5,.5,.2]]*3,
....:                                   [1/3]*3)
sage: G = m.graph(); G
Looped digraph on 3 vertices
sage: G.edges(sort=True)
[(0, 0, 0.3), (0, 2, 0.7), (1, 2, 1.0), (2, 0, 0.5), (2, 1, 0.5)]
sage: G.plot()
Graphics object consisting of 11 graphics primitives
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('.3'),Integer(0),
↪RealNumber('.7')],[Integer(0),Integer(0),Integer(1)],[RealNumber('.5'),
↪RealNumber('.5'),Integer(0)]],
...                                   [[RealNumber('.5'),RealNumber('.5'),
↪RealNumber('.2')]]*Integer(3),
```

(continues on next page)

(continued from previous page)

```
...                                     [Integer(1)/Integer(3)]*Integer(3))
>>> G = m.graph(); G
Looped digraph on 3 vertices
>>> G.edges(sort=True)
[(0, 0, 0.3), (0, 2, 0.7), (1, 2, 1.0), (2, 0, 0.5), (2, 1, 0.5)]
>>> G.plot()
Graphics object consisting of 11 graphics primitives
```

initial_probabilities()

Return the initial probabilities as a *TimeSeries* of length N, where N is the number of states of the Markov model.

EXAMPLES:

```
sage: m = hmm.DiscreteHiddenMarkovModel([[0.4,0.6],[0.1,0.9]],
....:                                   [[0.1,0.9],[0.5,0.5]],
....:                                   [.2,.8])
sage: pi = m.initial_probabilities(); pi
[0.2000, 0.8000]
sage: type(pi)
<... 'sage.stats.time_series.TimeSeries'>
```

```
>>> from sage.all import *
>>> m = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.4'),RealNumber('0.6')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [RealNumber('.2'),RealNumber('.8')])
>>> pi = m.initial_probabilities(); pi
[0.2000, 0.8000]
>>> type(pi)
<... 'sage.stats.time_series.TimeSeries'>
```

The returned time series is a copy, so changing it does not change the model:

```
sage: pi[0] = .1; pi[1] = .9
sage: m.initial_probabilities()
[0.2000, 0.8000]
```

```
>>> from sage.all import *
>>> pi[Integer(0)] = RealNumber('.1'); pi[Integer(1)] = RealNumber('.9')
>>> m.initial_probabilities()
[0.2000, 0.8000]
```

Some other models:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,1), (-1,1)],
....:                                   [.1,.9])
sage: m.initial_probabilities()
[0.1000, 0.9000]
sage: m = hmm.GaussianMixtureHiddenMarkovModel(
```

(continues on next page)

(continued from previous page)

```
....:          [[.9,.1],[.4,.6]],
....:          [[(.4,(0,1)), (.6,(1,0.1))], [(1,(0,1))]],
....:          [.7,.3])
sage: m.initial_probabilities()
[0.7000, 0.3000]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),Integer(1)), (-Integer(1),
↪Integer(1))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> m.initial_probabilities()
[0.1000, 0.9000]
>>> m = hmm.GaussianMixtureHiddenMarkovModel(
...          [[RealNumber('.9'),RealNumber('.1')],[RealNumber('.4'),RealNumber(
↪'.6')]],
...          [[(RealNumber('.4'),(Integer(0),Integer(1))), (RealNumber('.6'),
↪(Integer(1),RealNumber('0.1')))], [(Integer(1),(Integer(0),Integer(1)))]],
...          [RealNumber('.7'),RealNumber('.3')])
>>> m.initial_probabilities()
[0.7000, 0.3000]
```

sample(*length*, *number=None*, *starting_state=None*)

Return number samples from this HMM of given length.

INPUT:

- `length` – positive integer
- `number` – (default: `None`) if given, compute list of this many sample sequences
- `starting_state` – integer (or `None`); if specified, generate a sequence using this model starting with the given state instead of the initial probabilities to determine the starting state

OUTPUT:

- if `number` is not given, return a single *TimeSeries*.
- if `number` is given, return a list of *TimeSeries*.

EXAMPLES:

```
sage: set_random_seed(0)
sage: a = hmm.DiscreteHiddenMarkovModel([[0.1,0.9],[0.1,0.9]],
....:                                   [[1,0],[0,1]],
....:                                   [0,1])
sage: print(a.sample(10, 3))
[[1, 0, 1, 1, 1, 1, 0, 1, 1, 1],
 [1, 1, 0, 0, 1, 1, 1, 1, 1, 1],
 [1, 1, 1, 1, 0, 1, 0, 1, 1, 1]]
sage: a.sample(15)
[1, 1, 1, 1, 0 ... 1, 1, 1, 1, 1]
sage: a.sample(3, 1)
[[1, 1, 1]]
```

(continues on next page)

(continued from previous page)

```
sage: list(a.sample(1000)).count(0)
88
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.1'),RealNumber('0.9')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[Integer(1),Integer(0)],[Integer(0),
↪Integer(1)]],
...                                   [Integer(0),Integer(1)])
>>> print(a.sample(Integer(10), Integer(3)))
[[1, 0, 1, 1, 1, 1, 0, 1, 1, 1],
 [1, 1, 0, 0, 1, 1, 1, 1, 1, 1],
 [1, 1, 1, 1, 0, 1, 0, 1, 1, 1]]
>>> a.sample(Integer(15))
[1, 1, 1, 1, 0 ... 1, 1, 1, 1, 1]
>>> a.sample(Integer(3), Integer(1))
[[1, 1, 1]]
>>> list(a.sample(Integer(1000))).count(Integer(0))
88
```

If the emission symbols are set:

```
sage: set_random_seed(0)
sage: a = hmm.DiscreteHiddenMarkovModel([[0.5,0.5],[0.1,0.9]],
....:                                   [[1,0],[0,1]], [0,1],
....:                                   ['up', 'down'])
sage: a.sample(10)
['down', 'up', 'down', 'down', 'down', 'down', 'up', 'up', 'up', 'up']
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> a = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.5'),RealNumber('0.5')],
↪[RealNumber('0.1'),RealNumber('0.9')]],
...                                   [[Integer(1),Integer(0)],[Integer(0),
↪Integer(1)]], [Integer(0),Integer(1)],
...                                   ['up', 'down'])
>>> a.sample(Integer(10))
['down', 'up', 'down', 'down', 'down', 'down', 'up', 'up', 'up', 'up']
```

Force a starting state:

```
sage: set_random_seed(0); a.sample(10, starting_state=0)
['up', 'up', 'down', 'down', 'down', 'down', 'up', 'up', 'up', 'up']
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0)); a.sample(Integer(10), starting_
↪state=Integer(0))
['up', 'up', 'down', 'down', 'down', 'down', 'up', 'up', 'up', 'up']
```

transition_matrix()

Return the state transition matrix.

OUTPUT: a Sage matrix with real double precision (RDF) entries.

EXAMPLES:

```
sage: M = hmm.DiscreteHiddenMarkovModel([[0.7,0.3],[0.9,0.1]],
....:                                   [[0.5,.5],[.1,.9]],
....:                                   [0.3,0.7])
sage: T = M.transition_matrix(); T
[0.7 0.3]
[0.9 0.1]
```

```
>>> from sage.all import *
>>> M = hmm.DiscreteHiddenMarkovModel([[RealNumber('0.7'),RealNumber('0.3')],
↪[RealNumber('0.9'),RealNumber('0.1')]],
...                                   [[RealNumber('0.5'),RealNumber('.5')],
↪[RealNumber('.1'),RealNumber('.9')]],
...                                   [RealNumber('0.3'),RealNumber('0.7')])
>>> T = M.transition_matrix(); T
[0.7 0.3]
[0.9 0.1]
```

The returned matrix is mutable, but changing it does not change the transition matrix for the model:

```
sage: T[0,0] = .1; T[0,1] = .9
sage: M.transition_matrix()
[0.7 0.3]
[0.9 0.1]
```

```
>>> from sage.all import *
>>> T[Integer(0),Integer(0)] = RealNumber('.1'); T[Integer(0),Integer(1)] =␣
↪RealNumber('.9')
>>> M.transition_matrix()
[0.7 0.3]
[0.9 0.1]
```

Transition matrices for other types of models:

```
sage: M = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,1), (-1,1)],
....:                                   [.5,.5])
sage: M.transition_matrix()
[0.1 0.9]
[0.5 0.5]
sage: M = hmm.GaussianMixtureHiddenMarkovModel(
....:         [[.9,.1],[.4,.6]],
....:         [[(.4,(0,1)), (.6,(1,0.1))],[(1,(0,1))]],
....:         [.7,.3])
sage: M.transition_matrix()
[0.9 0.1]
[0.4 0.6]
```

```
>>> from sage.all import *
>>> M = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
```

(continues on next page)

(continued from previous page)

```
↪[RealNumber('.5'),RealNumber('.5')]],
...                                     [(Integer(1),Integer(1)), (-Integer(1),
↪Integer(1))],
...                                     [RealNumber('.5'),RealNumber('.5')])
>>> M.transition_matrix()
[0.1 0.9]
[0.5 0.5]
>>> M = hmm.GaussianMixtureHiddenMarkovModel(
...         [[RealNumber('.9'),RealNumber('.1')],[RealNumber('.4'),RealNumber(
↪'.6')]],
...         [[(RealNumber('.4'),(Integer(0),Integer(1))), (RealNumber('.6'),
↪(Integer(1),RealNumber('0.1')))],[(Integer(1),(Integer(0),Integer(1)))]],
...         [RealNumber('.7'),RealNumber('.3')])
>>> M.transition_matrix()
[0.9 0.1]
[0.4 0.6]
```

sage.stats.hmm.hmm.**unpickle_discrete_hmm_v0**(*A*, *B*, *pi*, *emission_symbols*, *name*)

sage.stats.hmm.hmm.**unpickle_discrete_hmm_v1**(*A*, *B*, *pi*, *n_out*, *emission_symbols*, *emission_symbols_dict*)

Return a *DiscreteHiddenMarkovModel*, restored from the arguments.

This function is used internally for unpickling.

CHAPTER

FIVE

CONTINUOUS EMISSION HIDDEN MARKOV MODELS

AUTHOR:

- William Stein, 2010-03

class sage.stats.hmm.chmm.**GaussianHiddenMarkovModel**

Bases: *HiddenMarkovModel*

Gaussian emissions Hidden Markov Model.

INPUT:

- `A` – matrix; the $N \times N$ transition matrix
- `B` – list of pairs `(mu, sigma)` that define the distributions
- `pi` – initial state probabilities
- `normalize` – boolean (default: `True`)

EXAMPLES:

We illustrate the primary functions with an example 2-state Gaussian HMM:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,1), (-1,1)],
....:                                   [.5,.5]); m
Gaussian Hidden Markov Model with 2 States
Transition matrix:
[0.1 0.9]
[0.5 0.5]
Emission parameters:
[(1.0, 1.0), (-1.0, 1.0)]
Initial probabilities: [0.5000, 0.5000]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),Integer(1)), (-Integer(1),
↪Integer(1))],
...                                   [RealNumber('.5'),RealNumber('.5')]); m
Gaussian Hidden Markov Model with 2 States
Transition matrix:
[0.1 0.9]
[0.5 0.5]
Emission parameters:
```

(continues on next page)

(continued from previous page)

```
[(1.0, 1.0), (-1.0, 1.0)]
Initial probabilities: [0.5000, 0.5000]
```

We query the defining transition matrix, emission parameters, and initial state probabilities:

```
sage: m.transition_matrix()
[0.1 0.9]
[0.5 0.5]
sage: m.emission_parameters()
[(1.0, 1.0), (-1.0, 1.0)]
sage: m.initial_probabilities()
[0.5000, 0.5000]
```

```
>>> from sage.all import *
>>> m.transition_matrix()
[0.1 0.9]
[0.5 0.5]
>>> m.emission_parameters()
[(1.0, 1.0), (-1.0, 1.0)]
>>> m.initial_probabilities()
[0.5000, 0.5000]
```

We obtain a sample sequence with 10 entries in it, and compute the logarithm of the probability of obtaining this sequence, given the model:

```
sage: obs = m.sample(5); obs  # random
[-1.6835, 0.0635, -2.1688, 0.3043, -0.3188]
sage: log_likelihood = m.log_likelihood(obs)
sage: counter = 0
sage: n = 0
sage: def add_samples(i):
....:     global counter, n
....:     for _ in range(i):
....:         n += 1
....:         obs2 = m.sample(5)
....:         if all(abs(obs2[i] - obs[i]) < 0.25 for i in range(5)):
....:             counter += 1

sage: add_samples(10000)
sage: while abs(log_likelihood - log(counter*1.0/n/0.5^5)) < 0.1:
....:     add_samples(10000)
```

```
>>> from sage.all import *
>>> obs = m.sample(Integer(5)); obs  # random
[-1.6835, 0.0635, -2.1688, 0.3043, -0.3188]
>>> log_likelihood = m.log_likelihood(obs)
>>> counter = Integer(0)
>>> n = Integer(0)
>>> def add_samples(i):
...     global counter, n
...     for _ in range(i):
...         n += Integer(1)
```

(continues on next page)

(continued from previous page)

```
...         obs2 = m.sample(Integer(5))
...         if all(abs(obs2[i] - obs[i]) < RealNumber('0.25') for i in␣
↪range(Integer(5))):
...             counter += Integer(1)

>>> add_samples(Integer(10000))
>>> while abs(log_likelihood - log(counter*RealNumber('1.0')/n/RealNumber('0.5
↪')**Integer(5))) < RealNumber('0.1'):
...     add_samples(Integer(10000))
```

We compute the Viterbi path, and probability that the given path of states produced obs:

```
sage: m.viterbi(obs)  # random
([1, 0, 1, 0, 1], -8.714092684611794)
```

```
>>> from sage.all import *
>>> m.viterbi(obs)  # random
([1, 0, 1, 0, 1], -8.714092684611794)
```

We use the Baum-Welch iterative algorithm to find another model for which our observation sequence is more likely:

```
sage: try:
....:     p, s = m.baum_welch(obs)
....:     assert p > log_likelihood
....:     assert (1 <= s <= 500)
....: except RuntimeError:
....:     pass
```

```
>>> from sage.all import *
>>> try:
...     p, s = m.baum_welch(obs)
...     assert p > log_likelihood
...     assert (Integer(1) <= s <= Integer(500))
... except RuntimeError:
...     pass
```

Notice that running Baum-Welch changed our model:

```
sage: m  # random
Gaussian Hidden Markov Model with 2 States
Transition matrix:
[   0.4154981366185841     0.584501863381416]
[   0.9999993174253741 6.825746258991804e-07]
Emission parameters:
[(0.4178882427119503, 0.5173109664360919),
 (-1.5025208631331122, 0.5085512836055119)]
Initial probabilities: [0.0000, 1.0000]
```

```
>>> from sage.all import *
>>> m  # random
Gaussian Hidden Markov Model with 2 States
```

(continues on next page)

(continued from previous page)

```
Transition matrix:
[   0.4154981366185841     0.584501863381416]
[   0.9999993174253741 6.825746258991804e-07]
Emission parameters:
[(0.4178882427119503, 0.5173109664360919),
 (-1.5025208631331122, 0.5085512836055119)]
Initial probabilities: [0.0000, 1.0000]
```

baum_welch(*obs*, *max_iter=500*, *log_likelihood_cutoff=0.0001*, *min_sd=0.01*, *fix_emissions=False*, *v=False*)

Given an observation sequence `obs`, improve this HMM using the Baum-Welch algorithm to increase the probability of observing `obs`.

INPUT:

- `obs` – a time series of emissions
- `max_iter` – integer (default: 500); maximum number of Baum-Welch steps to take
- `log_likelihood_cutoff` – positive float (default: 1e-4); the minimal improvement in likelihood with respect to the last iteration required to continue. Relative value to log likelihood.
- `min_sd` – positive float (default: 0.01); when reestimating, the standard deviation of emissions is not allowed to be less than `min_sd`.
- `fix_emissions` – boolean (default: `False`); if `True`, do not change emissions when updating

OUTPUT:

changes the model in place, and returns the log likelihood and number of iterations.

EXAMPLES:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: m.log_likelihood([-2,-1,.1,0.1])
-8.858282215986275
sage: m.baum_welch([-2,-1,.1,0.1])
(4.534646052182..., 7)
sage: m.log_likelihood([-2,-1,.1,0.1])
4.534646052182...
sage: m  # rel tol 3e-14
Gaussian Hidden Markov Model with 2 States
Transition matrix:
[   0.9999999992430161 7.569839394440382e-10]
[  0.49998462791192644    0.5000153720880736]
Emission parameters:
[(0.09999999999999999, 0.01), (-1.4999508147591902, 0.5000710504895474)]
Initial probabilities: [0.0000, 1.0000]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
```

(continues on next page)

(continued from previous page)

```
>>> m.log_likelihood([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.1
↪')])
-8.858282215986275
>>> m.baum_welch([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.1')])
(4.534646052182..., 7)
>>> m.log_likelihood([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.1
↪')])
4.534646052182...
>>> m  # rel tol 3e-14
Gaussian Hidden Markov Model with 2 States
Transition matrix:
[   0.9999999992430161 7.569839394440382e-10]
[  0.49998462791192644    0.5000153720880736]
Emission parameters:
[(0.09999999999999999, 0.01), (-1.4999508147591902, 0.5000710504895474)]
Initial probabilities: [0.0000, 1.0000]
```

We illustrate bounding the standard deviation below. Note that above we had different emission parameters when the `min_sd` was the default of 0.01:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: m.baum_welch([-2,-1,.1,0.1], min_sd=1)
(-4.07939572755..., 32)
sage: m.emission_parameters()
[(-0.2663018798..., 1.0), (-1.99850979..., 1.0)]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> m.baum_welch([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.1')],
↪ min_sd=Integer(1))
(-4.07939572755..., 32)
>>> m.emission_parameters()
[(-0.2663018798..., 1.0), (-1.99850979..., 1.0)]
```

We watch the log likelihoods of the model converge, step by step. We set the random seed beforehand so that the output is predictable (and guaranteed to avoid random floating point issues):

```
sage: set_random_seed(0)
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: v = m.sample(10)
sage: l = stats.TimeSeries([m.baum_welch(v, max_iter=1)[0]
....:                       for _ in range(len(v))])
sage: all(l[i] <= l[i+1] + 0.0001 for i in range(9))
True
```

(continues on next page)

(continued from previous page)

```
sage: l
[-20.1167, -17.7611, -16.9814, -16.9364, -16.9314,
 -16.9309, -16.9309, -16.9309, -16.9309, -16.9309]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> v = m.sample(Integer(10))
>>> l = stats.TimeSeries([m.baum_welch(v, max_iter=Integer(1))[Integer(0)]
...                       for _ in range(len(v))])
>>> all(l[i] <= l[i+Integer(1)] + RealNumber('0.0001') for i in␣
↪range(Integer(9)))
True
>>> l
[-20.1167, -17.7611, -16.9814, -16.9364, -16.9314,
 -16.9309, -16.9309, -16.9309, -16.9309, -16.9309]
```

We illustrate fixing emissions; again the random seed is fixed to ensure that the output is what we expect it to be:

```
sage: set_random_seed(0)
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.9,.1]],
....:                                   [(1,2),(-1,.5)],
....:                                   [.3,.7])
sage: v = m.sample(100)
sage: m.baum_welch(v,fix_emissions=True)
(-164.72944548204..., 23)
sage: m.emission_parameters()
[(1.0, 2.0), (-1.0, 0.5)]
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.9,.1]],
....:                                   [(1,2),(-1,.5)],
....:                                   [.3,.7])
sage: m.baum_welch(v)
(-162.854370397998..., 49)
sage: m.emission_parameters()  # rel tol 3e-14
[(1.2722419172602375, 2.371368751761901),
 (-0.9486174675179113, 0.5762360385123765)]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.9'),RealNumber('.1')]],
...                                   [(Integer(1),Integer(2)),(-Integer(1),
↪RealNumber('.5'))],
...                                   [RealNumber('.3'),RealNumber('.7')])
>>> v = m.sample(Integer(100))
>>> m.baum_welch(v,fix_emissions=True)
(-164.72944548204..., 23)
```

(continues on next page)

(continued from previous page)

```
>>> m.emission_parameters()
[(1.0, 2.0), (-1.0, 0.5)]
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.9'),RealNumber('.1')]],
...                                    [(Integer(1),Integer(2)),(-Integer(1),
↪RealNumber('.5'))],
...                                    [RealNumber('.3'),RealNumber('.7')])
>>> m.baum_welch(v)
(-162.854370397998..., 49)
>>> m.emission_parameters()  # rel tol 3e-14
[(1.2722419172602375, 2.371368751761901),
 (-0.9486174675179113, 0.5762360385123765)]
```

emission_parameters()

Return the parameters that define the normal distributions associated to all of the states.

OUTPUT:

a list `B` of pairs `B[i] = (mu, std)`, such that the distribution associated to state i is normal with mean `mu` and standard deviation `std`.

EXAMPLES:

```
sage: M = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: M.emission_parameters()
[(1.0, 0.5), (-1.0, 3.0)]
```

```
>>> from sage.all import *
>>> M = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> M.emission_parameters()
[(1.0, 0.5), (-1.0, 3.0)]
```

generate_sequence(*length*, *starting_state=None*)

Return a sample of the given length from this HMM.

INPUT:

- `length` – positive integer
- `starting_state` – integer (or `None`); if specified then generate a sequence using this model starting with the given state instead of the initial probabilities to determine the starting state.

OUTPUT:

- an *IntList* or list of emission symbols
- *TimeSeries* of emissions

EXAMPLES:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: m.generate_sequence(5)  # random
([-3.0505, 0.5317, -4.5065, 0.6521, 1.0435], [1, 0, 1, 0, 1])
sage: m.generate_sequence(0)
([], [])
sage: m.generate_sequence(-1)
Traceback (most recent call last):
...
ValueError: length must be nonnegative
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> m.generate_sequence(Integer(5))  # random
([-3.0505, 0.5317, -4.5065, 0.6521, 1.0435], [1, 0, 1, 0, 1])
>>> m.generate_sequence(Integer(0))
([], [])
>>> m.generate_sequence(-Integer(1))
Traceback (most recent call last):
...
ValueError: length must be nonnegative
```

Verify numerically that the starting state is 0 with probability about 0.1:

```
sage: counter = 0
sage: n = 0
sage: def add_samples(i):
....:     global counter, n
....:     for i in range(i):
....:         n += 1
....:         if m.generate_sequence(1)[1][0] == 0:
....:             counter += 1

sage: add_samples(10^5)
sage: while abs(counter*1.0 / n - 0.1) > 0.01: add_samples(10^5)
```

```
>>> from sage.all import *
>>> counter = Integer(0)
>>> n = Integer(0)
>>> def add_samples(i):
...     global counter, n
...     for i in range(i):
...         n += Integer(1)
...         if m.generate_sequence(Integer(1))[Integer(1)][Integer(0)] ==␣
↪Integer(0):
...             counter += Integer(1)

```

(continues on next page)

(continued from previous page)

```
>>> add_samples(Integer(10)**Integer(5))
>>> while abs(counter*RealNumber('1.0') / n - RealNumber('0.1')) > RealNumber(
↪'0.01'): add_samples(Integer(10)**Integer(5))
```

Example in which the starting state is 0 (see Issue #11452):

```
sage: set_random_seed(23);  m.generate_sequence(2)
([0.6501, -2.0151], [0, 1])
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(23));  m.generate_sequence(Integer(2))
([0.6501, -2.0151], [0, 1])
```

Force a starting state of 1 even though as we saw above it would be 0:

```
sage: set_random_seed(23);  m.generate_sequence(2, starting_state=1)
([-3.1491, -1.0244], [1, 1])
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(23));  m.generate_sequence(Integer(2), starting_
↪state=Integer(1))
([-3.1491, -1.0244], [1, 1])
```

log_likelihood(*obs*)

Return the logarithm of a continuous analogue of the probability that this model produced the given observation sequence.

Note that the "continuous analogue of the probability" above can be bigger than 1, hence the logarithm can be positive.

INPUT:

- `obs` – sequence of observations

OUTPUT: float

EXAMPLES:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: m.log_likelihood([1,1,1])
-4.297880766072486
sage: s = m.sample(20)
sage: -80 < m.log_likelihood(s) < -20
True
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> m.log_likelihood([Integer(1),Integer(1),Integer(1)])
```

(continues on next page)

(continued from previous page)

```
-4.297880766072486
>>> s = m.sample(Integer(20))
>>> -Integer(80) < m.log_likelihood(s) < -Integer(20)
True
```

viterbi(*obs*)

Determine "the" hidden sequence of states that is most likely to produce the given sequence `obs` of observations, along with the probability that this hidden sequence actually produced the observation.

INPUT:

- `obs` – sequence of emitted ints or symbols

OUTPUT:

- `list` – "the" most probable sequence of hidden states, i.e., the Viterbi path
- `float` – log of probability that the observed sequence was produced by the Viterbi sequence of states

EXAMPLES:

We find the optimal state sequence for a given model:

```
sage: m = hmm.GaussianHiddenMarkovModel([[0.5,0.5],[0.5,0.5]],
....:                                   [(0,1),(10,1)],
....:                                   [0.5,0.5])
sage: m.viterbi([0,1,10,10,1])
([0, 0, 1, 1, 0], -9.0604285688230...)
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('0.5'),RealNumber('0.5')],
↪[RealNumber('0.5'),RealNumber('0.5')]],
...                                   [(Integer(0),Integer(1)),(Integer(10),
↪Integer(1))],
...                                   [RealNumber('0.5'),RealNumber('0.5')])
>>> m.viterbi([Integer(0),Integer(1),Integer(10),Integer(10),Integer(1)])
([0, 0, 1, 1, 0], -9.0604285688230...)
```

Another example in which the most likely states change based on the last observation:

```
sage: m = hmm.GaussianHiddenMarkovModel([[.1,.9],[.5,.5]],
....:                                   [(1,.5), (-1,3)],
....:                                   [.1,.9])
sage: m.viterbi([-2,-1,.1,0.1])
([1, 1, 0, 1], -9.61823698847639...)
sage: m.viterbi([-2,-1,.1,0.3])
([1, 1, 1, 0], -9.566023653378513)
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[RealNumber('.1'),RealNumber('.9')],
↪[RealNumber('.5'),RealNumber('.5')]],
...                                   [(Integer(1),RealNumber('.5')), (-
↪Integer(1),Integer(3))],
...                                   [RealNumber('.1'),RealNumber('.9')])
>>> m.viterbi([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.1')])
```

(continues on next page)

(continued from previous page)

```
([1, 1, 0, 1], -9.61823698847639...)
>>> m.viterbi([-Integer(2),-Integer(1),RealNumber('.1'),RealNumber('0.3')])
([1, 1, 1, 0], -9.566023653378513)
```

class sage.stats.hmm.chmm.**GaussianMixtureHiddenMarkovModel**

Bases: *GaussianHiddenMarkovModel*

Gaussian mixture Hidden Markov Model.

INPUT:

- `A` – matrix; the $N \times N$ transition matrix
- `B` – list of mixture definitions for each state. Each state may have a varying number of gaussians with selection probabilities that sum to 1 and encoded as `(p, (mu,sigma))`
- `pi` – initial state probabilities
- `normalize` – boolean (default: `True`); if given, input is normalized to define valid probability distributions, e.g., the entries of A are made nonnegative and the rows sum to 1, and the probabilities in `pi` are normalized.

EXAMPLES:

```
sage: A = [[0.5,0.5],[0.5,0.5]]
sage: B = [[(0.9,(0.0,1.0)), (0.1,(1,10000))],[(1,(1,1)), (0,(0,0.1))]]
sage: hmm.GaussianMixtureHiddenMarkovModel(A, B, [1,0])
Gaussian Mixture Hidden Markov Model with 2 States
Transition matrix:
[0.5 0.5]
[0.5 0.5]
Emission parameters:
[0.9*N(0.0,1.0) + 0.1*N(1.0,10000.0), 1.0*N(1.0,1.0) + 0.0*N(0.0,0.1)]
Initial probabilities: [1.0000, 0.0000]
```

```
>>> from sage.all import *
>>> A = [[RealNumber('0.5'),RealNumber('0.5')],[RealNumber('0.5'),RealNumber('0.5
↪')]]
>>> B = [[(RealNumber('0.9'),(RealNumber('0.0'),RealNumber('1.0'))), (RealNumber(
↪'0.1'),(Integer(1),Integer(10000)))],[(Integer(1),(Integer(1),Integer(1))),␣
↪(Integer(0),(Integer(0),RealNumber('0.1')))]]
>>> hmm.GaussianMixtureHiddenMarkovModel(A, B, [Integer(1),Integer(0)])
Gaussian Mixture Hidden Markov Model with 2 States
Transition matrix:
[0.5 0.5]
[0.5 0.5]
Emission parameters:
[0.9*N(0.0,1.0) + 0.1*N(1.0,10000.0), 1.0*N(1.0,1.0) + 0.0*N(0.0,0.1)]
Initial probabilities: [1.0000, 0.0000]
```

baum_welch(*obs*, *max_iter=1000*, *log_likelihood_cutoff=1e-12*, *min_sd=0.01*, *fix_emissions=False*)

Given an observation sequence `obs`, improve this HMM using the Baum-Welch algorithm to increase the probability of observing `obs`.

INPUT:

- `obs` – a time series of emissions

- `max_iter` – integer (default: 1000); maximum number of Baum-Welch steps to take
- `log_likelihood_cutoff` – positive float (default: 1e-12); the minimal improvement in likelihood with respect to the last iteration required to continue. Relative value to log likelihood.
- `min_sd` – positive float (default: 0.01); when reestimating, the standard deviation of emissions is not allowed to be less than `min_sd`.
- `fix_emissions` – boolean (default: `False`); if `True`, do not change emissions when updating

OUTPUT:

changes the model in place, and returns the log likelihood and number of iterations.

EXAMPLES:

```
sage: m = hmm.GaussianMixtureHiddenMarkovModel(
....:         [[.9,.1],[.4,.6]],
....:         [[(.4,(0,1)), (.6,(1,0.1))], [(1,(0,1))]],
....:         [.7,.3])
sage: set_random_seed(0); v = m.sample(10); v
[0.3576, -0.9365, 0.9449, -0.6957, 1.0217,
 0.9644, 0.9987, -0.5950, -1.0219, 0.6477]
sage: m.log_likelihood(v)
-8.31408655939536...
sage: m.baum_welch(v)
(2.18905068682..., 15)
sage: m.log_likelihood(v)
2.18905068682...
sage: m  # rel tol 6e-12
Gaussian Mixture Hidden Markov Model with 2 States
Transition matrix:
[   0.8746363339773399   0.12536366602266016]
[                  1.0 1.451685202290174e-40]
Emission parameters:
[0.500161629343*N(-0.812298726239,0.173329026744)
  + 0.499838370657*N(0.982433690378,0.029719932009),
 1.0*N(0.503260056832,0.145881515324)]
Initial probabilities: [0.0000, 1.0000]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianMixtureHiddenMarkovModel(
...         [[RealNumber('.9'),RealNumber('.1')],[RealNumber('.4'),RealNumber(
↪'.6')]],
...         [[(RealNumber('.4'),(Integer(0),Integer(1))), (RealNumber('.6'),
↪(Integer(1),RealNumber('0.1')))], [(Integer(1),(Integer(0),Integer(1)))]],
...         [RealNumber('.7'),RealNumber('.3')])
>>> set_random_seed(Integer(0)); v = m.sample(Integer(10)); v
[0.3576, -0.9365, 0.9449, -0.6957, 1.0217,
 0.9644, 0.9987, -0.5950, -1.0219, 0.6477]
>>> m.log_likelihood(v)
-8.31408655939536...
>>> m.baum_welch(v)
(2.18905068682..., 15)
>>> m.log_likelihood(v)
2.18905068682...
```

(continues on next page)

(continued from previous page)

```
>>> m  # rel tol 6e-12
Gaussian Mixture Hidden Markov Model with 2 States
Transition matrix:
[   0.8746363339773399   0.12536366602266016]
[                  1.0 1.451685202290174e-40]
Emission parameters:
[0.500161629343*N(-0.812298726239,0.173329026744)
  + 0.499838370657*N(0.982433690378,0.029719932009),
 1.0*N(0.503260056832,0.145881515324)]
Initial probabilities: [0.0000, 1.0000]
```

We illustrate bounding the standard deviation below. Note that above we had different emission parameters when the min_sd was the default of 0.01:

```
sage: m = hmm.GaussianMixtureHiddenMarkovModel(
....:         [[.9,.1],[.4,.6]],
....:         [[(.4,(0,1)), (.6,(1,0.1))], [(1,(0,1))]],
....:         [.7,.3])
sage: m.baum_welch(v, min_sd=1)
(-12.617885761692..., 1000)
sage: m.emission_parameters()  # rel tol 6e-12
[0.503545634447*N(0.200166509595,1.0) + 0.496454365553*N(0.200166509595,1.0),
 1.0*N(0.0543433426535,1.0)]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianMixtureHiddenMarkovModel(
...         [[RealNumber('.9'),RealNumber('.1')],[RealNumber('.4'),RealNumber(
↪'.6')]],
...         [[(RealNumber('.4'),(Integer(0),Integer(1))), (RealNumber('.6'),
↪(Integer(1),RealNumber('0.1')))], [(Integer(1),(Integer(0),Integer(1)))]],
...         [RealNumber('.7'),RealNumber('.3')])
>>> m.baum_welch(v, min_sd=Integer(1))
(-12.617885761692..., 1000)
>>> m.emission_parameters()  # rel tol 6e-12
[0.503545634447*N(0.200166509595,1.0) + 0.496454365553*N(0.200166509595,1.0),
 1.0*N(0.0543433426535,1.0)]
```

We illustrate fixing all emissions:

```
sage: m = hmm.GaussianMixtureHiddenMarkovModel(
....:         [[.9,.1],[.4,.6]],
....:         [[(.4,(0,1)), (.6,(1,0.1))], [(1,(0,1))]],
....:         [.7,.3])
sage: set_random_seed(0); v = m.sample(10)
sage: m.baum_welch(v, fix_emissions=True)
(-7.58656858997..., 36)
sage: m.emission_parameters()
[0.4*N(0.0,1.0) + 0.6*N(1.0,0.1),
 1.0*N(0.0,1.0)]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianMixtureHiddenMarkovModel(
```

(continues on next page)

(continued from previous page)

```
...           [[RealNumber('.9'),RealNumber('.1')],[RealNumber('.4'),RealNumber(
↪'.6')]],
...           [[(RealNumber('.4'),(Integer(0),Integer(1))), (RealNumber('.6'),
↪(Integer(1),RealNumber('0.1')))], [(Integer(1),(Integer(0),Integer(1)))]],
...           [RealNumber('.7'),RealNumber('.3')])
>>> set_random_seed(Integer(0)); v = m.sample(Integer(10))
>>> m.baum_welch(v, fix_emissions=True)
(-7.58656858997..., 36)
>>> m.emission_parameters()
[0.4*N(0.0,1.0) + 0.6*N(1.0,0.1),
 1.0*N(0.0,1.0)]
```

emission_parameters()

Return a list of all the emission distributions.

OUTPUT: list of Gaussian mixtures

EXAMPLES:

```
sage: m = hmm.GaussianMixtureHiddenMarkovModel([[.9,.1],[.4,.6]],
....:                                          [[(.4,(0,1)), (.6,(1,0.1))],␣
↪[(1,(0,1))]],
....:                                          [.7,.3])
sage: m.emission_parameters()
[0.4*N(0.0,1.0) + 0.6*N(1.0,0.1), 1.0*N(0.0,1.0)]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianMixtureHiddenMarkovModel([[RealNumber('.9'),RealNumber('.1
↪')],[RealNumber('.4'),RealNumber('.6')]],
...                                          [[(RealNumber('.4'),(Integer(0),
↪Integer(1))), (RealNumber('.6'),(Integer(1),RealNumber('0.1')))],␣
↪[(Integer(1),(Integer(0),Integer(1)))]],
...                                          [RealNumber('.7'),RealNumber('.3
↪')])
>>> m.emission_parameters()
[0.4*N(0.0,1.0) + 0.6*N(1.0,0.1), 1.0*N(0.0,1.0)]
```

sage.stats.hmm.chmm.**unpickle_gaussian_hmm_v0**(*A*, *B*, *pi*, *name*)

EXAMPLES:

```
sage: m = hmm.GaussianHiddenMarkovModel([[1]], [(0,1)], [1])
sage: sage.stats.hmm.chmm.unpickle_gaussian_hmm_v0(m.transition_matrix(), m.
↪emission_parameters(), m.initial_probabilities(), 'test')
Gaussian Hidden Markov Model with 1 States
Transition matrix:
[1.0]
Emission parameters:
[(0.0, 1.0)]
Initial probabilities: [1.0000]
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[Integer(1)]], [(Integer(0),Integer(1))],␣
```

(continues on next page)

(continued from previous page)

```
↪[Integer(1)])
>>> sage.stats.hmm.chmm.unpickle_gaussian_hmm_v0(m.transition_matrix(), m.
↪emission_parameters(), m.initial_probabilities(), 'test')
Gaussian Hidden Markov Model with 1 States
Transition matrix:
[1.0]
Emission parameters:
[(0.0, 1.0)]
Initial probabilities: [1.0000]
```

sage.stats.hmm.chmm.**unpickle_gaussian_hmm_v1**(*A*, *B*, *pi*, *prob*, *n_out*)

EXAMPLES:

```
sage: m = hmm.GaussianHiddenMarkovModel([[1]], [(0,1)], [1])
sage: loads(dumps(m)) == m   # indirect test
True
```

```
>>> from sage.all import *
>>> m = hmm.GaussianHiddenMarkovModel([[Integer(1)]], [(Integer(0),Integer(1))],␣
↪[Integer(1)])
>>> loads(dumps(m)) == m   # indirect test
True
```

sage.stats.hmm.chmm.**unpickle_gaussian_mixture_hmm_v1**(*A*, *B*, *pi*, *mixture*)

EXAMPLES:

```
sage: m = hmm.GaussianMixtureHiddenMarkovModel([[1]], [[(.4,(0,1)), (.6,(1,0.
↪1))]], [1])
sage: loads(dumps(m)) == m   # indirect test
True
```

```
>>> from sage.all import *
>>> m = hmm.GaussianMixtureHiddenMarkovModel([[Integer(1)]], [[(RealNumber('.4'),
↪(Integer(0),Integer(1))), (RealNumber('.6'),(Integer(1),RealNumber('0.1')))]],␣
↪[Integer(1)])
>>> loads(dumps(m)) == m   # indirect test
True
```

CHAPTER

SIX

DISTRIBUTIONS USED IN IMPLEMENTING HIDDEN MARKOV MODELS

These distribution classes are designed specifically for HMM's and not for general use in statistics. For example, they have fixed or non-fixed status, which only make sense relative to being used in a hidden Markov model.

AUTHOR:

- William Stein, 2010-03

class sage.stats.hmm.distributions.**DiscreteDistribution**

Bases: *Distribution*

class sage.stats.hmm.distributions.**Distribution**

Bases: object

A distribution.

plot(**args*, ***kwds*)

Return a plot of the probability density function.

INPUT:

- args and kwds, passed to the Sage *plot()* function

OUTPUT: a Graphics object

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.plot(-10,30)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.plot(-Integer(10),Integer(30))
Graphics object consisting of 1 graphics primitive
```

prob(x)

The probability density function evaluated at x.

INPUT:

- x – object

OUTPUT: float

EXAMPLES:

This method must be defined in a derived class:

```
sage: import sage.stats.hmm.distributions
sage: sage.stats.hmm.distributions.Distribution().prob(0)
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> import sage.stats.hmm.distributions
>>> sage.stats.hmm.distributions.Distribution().prob(Integer(0))
Traceback (most recent call last):
...
NotImplementedError
```

sample(*n=None*)

Return either a single sample (the default) or n samples from this probability distribution.

INPUT:

- `n` – `None` or a positive integer

OUTPUT: a single sample if n is 1; otherwise many samples

EXAMPLES:

This method must be defined in a derived class:

```
sage: import sage.stats.hmm.distributions
sage: sage.stats.hmm.distributions.Distribution().sample()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> import sage.stats.hmm.distributions
>>> sage.stats.hmm.distributions.Distribution().sample()
Traceback (most recent call last):
...
NotImplementedError
```

class sage.stats.hmm.distributions.**GaussianDistribution**

Bases: *Distribution*

class sage.stats.hmm.distributions.**GaussianMixtureDistribution**

Bases: *Distribution*

A probability distribution defined by taking a weighted linear combination of Gaussian distributions.

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.3,1,2),(.7,-1,1)]); P
0.3*N(1.0,2.0) + 0.7*N(-1.0,1.0)
```

(continues on next page)

(continued from previous page)

```
sage: P[0]
(0.3, 1.0, 2.0)
sage: P.is_fixed()
False
sage: P.fix(1)
sage: P.is_fixed(0)
False
sage: P.is_fixed(1)
True
sage: P.unfix(1)
sage: P.is_fixed(1)
False
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.3'),Integer(1),Integer(2)),
↪(RealNumber('.7'),-Integer(1),Integer(1))]); P
0.3*N(1.0,2.0) + 0.7*N(-1.0,1.0)
>>> P[Integer(0)]
(0.3, 1.0, 2.0)
>>> P.is_fixed()
False
>>> P.fix(Integer(1))
>>> P.is_fixed(Integer(0))
False
>>> P.is_fixed(Integer(1))
True
>>> P.unfix(Integer(1))
>>> P.is_fixed(Integer(1))
False
```

fix(*i=None*)

Set that this *GaussianMixtureDistribution* (or its i-th component) is fixed when using Baum-Welch to update the corresponding HMM.

INPUT:

- `i` – `None` (default) or integer; if given, only fix the i-th component

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.fix(1); P.is_fixed()
False
sage: P.is_fixed(1)
True
sage: P.fix(); P.is_fixed()
True
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.fix(Integer(1)); P.is_fixed()
```

(continues on next page)

(continued from previous page)

```
False
>>> P.is_fixed(Integer(1))
True
>>> P.fix(); P.is_fixed()
True
```

is_fixed(*i=None*)

Return whether or not this *GaussianMixtureDistribution* is fixed when using Baum-Welch to update the corresponding HMM.

INPUT:

- i – None (default) or integer; if given, only return whether the i-th component is fixed

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.is_fixed()
False
sage: P.is_fixed(0)
False
sage: P.fix(0); P.is_fixed()
False
sage: P.is_fixed(0)
True
sage: P.fix(); P.is_fixed()
True
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.is_fixed()
False
>>> P.is_fixed(Integer(0))
False
>>> P.fix(Integer(0)); P.is_fixed()
False
>>> P.is_fixed(Integer(0))
True
>>> P.fix(); P.is_fixed()
True
```

prob(x)

Return the probability of x.

Since this is a continuous distribution, this is defined to be the limit of the p's such that the probability of [x,x+h] is p*h.

INPUT:

- x – float

OUTPUT: float

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.prob(.5)
0.21123919605857971
sage: P.prob(-100)
0.0
sage: P.prob(20)
0.1595769121605731
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.prob(RealNumber('.5'))
0.21123919605857971
>>> P.prob(-Integer(100))
0.0
>>> P.prob(Integer(20))
0.1595769121605731
```

prob_m(*x*, *m*)

Return the probability of x using just the m-th summand.

INPUT:

- `x` – float
- `m` – integer

OUTPUT: float

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.prob_m(.5, 0)
2.7608117680508...e-97
sage: P.prob_m(.5, 1)
0.21123919605857971
sage: P.prob_m(.5, 2)
0.0
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.prob_m(RealNumber('.5'), Integer(0))
2.7608117680508...e-97
>>> P.prob_m(RealNumber('.5'), Integer(1))
0.21123919605857971
>>> P.prob_m(RealNumber('.5'), Integer(2))
0.0
```

sample(*n=None*)

Return a single sample from this distribution (by default), or if $n > 1$, return a *TimeSeries* of samples.

INPUT:

- `n` – integer or `None` (default: `None`)

OUTPUT:

- float if `n` is `None` (default); otherwise a *TimeSeries*

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: type(P.sample())
<class 'float'>
sage: l = P.sample(1)
sage: len(l)
1
sage: type(l)
<class 'sage.stats.time_series.TimeSeries'>
sage: l = P.sample(5)
sage: len(l)
5
sage: type(l)
<class 'sage.stats.time_series.TimeSeries'>
sage: l = P.sample(0)
sage: len(l)
0
sage: type(l)
<class 'sage.stats.time_series.TimeSeries'>
sage: P.sample(-3)
Traceback (most recent call last):
...
ValueError: n must be nonnegative
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> type(P.sample())
<class 'float'>
>>> l = P.sample(Integer(1))
>>> len(l)
1
>>> type(l)
<class 'sage.stats.time_series.TimeSeries'>
>>> l = P.sample(Integer(5))
>>> len(l)
5
>>> type(l)
<class 'sage.stats.time_series.TimeSeries'>
>>> l = P.sample(Integer(0))
>>> len(l)
0
>>> type(l)
<class 'sage.stats.time_series.TimeSeries'>
>>> P.sample(-Integer(3))
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: n must be nonnegative
```

unfix(*i=None*)

Set that this *GaussianMixtureDistribution* (or its i-th component) is not fixed when using Baum-Welch to update the corresponding HMM.

INPUT:

- `i` – `None` (default) or integer; if given, only fix the i-th component

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: P.fix(1); P.is_fixed(1)
True
sage: P.unfix(1); P.is_fixed(1)
False
sage: P.fix(); P.is_fixed()
True
sage: P.unfix(); P.is_fixed()
False
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2
↪'),Integer(20),RealNumber('.5'))])
>>> P.fix(Integer(1)); P.is_fixed(Integer(1))
True
>>> P.unfix(Integer(1)); P.is_fixed(Integer(1))
False
>>> P.fix(); P.is_fixed()
True
>>> P.unfix(); P.is_fixed()
False
```

sage.stats.hmm.distributions.**unpickle_gaussian_mixture_distribution_v1**(*c0*, *c1*, *param*, *fixed*)

Used in unpickling *GaussianMixtureDistribution* objects.

EXAMPLES:

```
sage: P = hmm.GaussianMixtureDistribution([(.2,-10,.5),(.6,1,1),(.2,20,.5)])
sage: loads(dumps(P)) == P          # indirect doctest
True
```

```
>>> from sage.all import *
>>> P = hmm.GaussianMixtureDistribution([(RealNumber('.2'),-Integer(10),
↪RealNumber('.5')),(RealNumber('.6'),Integer(1),Integer(1)),(RealNumber('.2'),
↪Integer(20),RealNumber('.5'))])
>>> loads(dumps(P)) == P          # indirect doctest
True
```

CHAPTER

SEVEN

HIDDEN MARKOV MODELS – UTILITY FUNCTIONS

AUTHOR:

- William Stein, 2010-03

class sage.stats.hmm.util.**HMM_Util**

Bases: object

A class used in order to share cdef's methods between different files.

initial_probs_to_TimeSeries(*pi*, *normalize*)

This function is used internally by the __init__ methods of various Hidden Markov Models.

INPUT:

- `pi` – vector, list, or *TimeSeries*
- `normalize` – if `True`, replace negative entries by 0 and rescale to ensure that the sum of the entries in each row is equal to 1. If the sum of the entries in a row is 0, replace them all by $1/N$.

OUTPUT: a *TimeSeries* of length N

EXAMPLES:

```
sage: import sage.stats.hmm.util
sage: u = sage.stats.hmm.util.HMM_Util()
sage: u.initial_probs_to_TimeSeries([0.1,0.2,0.9], True)
[0.0833, 0.1667, 0.7500]
sage: u.initial_probs_to_TimeSeries([0.1,0.2,0.9], False)
[0.1000, 0.2000, 0.9000]
```

```
>>> from sage.all import *
>>> import sage.stats.hmm.util
>>> u = sage.stats.hmm.util.HMM_Util()
>>> u.initial_probs_to_TimeSeries([RealNumber('0.1'),RealNumber('0.2'),
↪RealNumber('0.9')], True)
[0.0833, 0.1667, 0.7500]
>>> u.initial_probs_to_TimeSeries([RealNumber('0.1'),RealNumber('0.2'),
↪RealNumber('0.9')], False)
[0.1000, 0.2000, 0.9000]
```

normalize_probability_TimeSeries(T, i, j)

This function is used internally by the Hidden Markov Models code.

Replace entries of T[i:j] in place so that they are all nonnegative and sum to 1. Negative entries are replaced by 0 and T[i:j] is then rescaled to ensure that the sum of the entries in each row is equal to 1. If all entries are 0, replace them by 1/(j-i).

INPUT:

- `T` – a *TimeSeries*
- `i` – nonnegative integer
- `j` – nonnegative integer

OUTPUT: `T` is modified

EXAMPLES:

```
sage: import sage.stats.hmm.util
sage: T = stats.TimeSeries([.1, .3, .7, .5])
sage: u = sage.stats.hmm.util.HMM_Util()
sage: u.normalize_probability_TimeSeries(T,0,3)
sage: T
[0.0909, 0.2727, 0.6364, 0.5000]
sage: u.normalize_probability_TimeSeries(T,0,4)
sage: T
[0.0606, 0.1818, 0.4242, 0.3333]
sage: abs(T.sum()-1) < 1e-8    # might not exactly equal 1 due to rounding
True
```

```
>>> from sage.all import *
>>> import sage.stats.hmm.util
>>> T = stats.TimeSeries([RealNumber('.1'), RealNumber('.3'), RealNumber('.7
↪'), RealNumber('.5')])
>>> u = sage.stats.hmm.util.HMM_Util()
>>> u.normalize_probability_TimeSeries(T,Integer(0),Integer(3))
>>> T
[0.0909, 0.2727, 0.6364, 0.5000]
>>> u.normalize_probability_TimeSeries(T,Integer(0),Integer(4))
>>> T
[0.0606, 0.1818, 0.4242, 0.3333]
>>> abs(T.sum()-Integer(1)) < RealNumber('1e-8')    # might not exactly equal␣
↪1 due to rounding
True
```

state_matrix_to_TimeSeries(*A*, *N*, *normalize*)

This function is used internally by the `__init__` methods of Hidden Markov Models to make a transition matrix from `A`.

INPUT:

- `A` – matrix, list, list of lists, or *TimeSeries*
- `N` – number of states
- `normalize` – if `True`, replace negative entries by 0 and rescale to ensure that the sum of the entries in each row is equal to 1. If the sum of the entries in a row is 0, replace them all by $1/N$.

OUTPUT: a *TimeSeries*

EXAMPLES:

```
sage: import sage.stats.hmm.util
sage: u = sage.stats.hmm.util.HMM_Util()
sage: u.state_matrix_to_TimeSeries([[.1,.7],[3/7,4/7]], 2, True)
[0.1250, 0.8750, 0.4286, 0.5714]
sage: u.state_matrix_to_TimeSeries([[.1,.7],[3/7,4/7]], 2, False)
[0.1000, 0.7000, 0.4286, 0.5714]
```

```
>>> from sage.all import *
>>> import sage.stats.hmm.util
>>> u = sage.stats.hmm.util.HMM_Util()
>>> u.state_matrix_to_TimeSeries([[RealNumber('.1'),RealNumber('.7')],
↪[Integer(3)/Integer(7),Integer(4)/Integer(7)]], Integer(2), True)
[0.1250, 0.8750, 0.4286, 0.5714]
>>> u.state_matrix_to_TimeSeries([[RealNumber('.1'),RealNumber('.7')],
↪[Integer(3)/Integer(7),Integer(4)/Integer(7)]], Integer(2), False)
[0.1000, 0.7000, 0.4286, 0.5714]
```

CHAPTER

EIGHT

DISCRETE GAUSSIAN SAMPLERS OVER THE INTEGERS

This class realizes oracles which returns integers proportionally to $\exp(-(x-c)^2/(2^2))$. All oracles are implemented using rejection sampling. See *DiscreteGaussianDistributionIntegerSampler.__init__()* for which algorithms are available.

AUTHORS:

- Martin Albrecht (2014-06-28): initial version

EXAMPLES:

We construct a sampler for the distribution $D_{3,c}$ with width $= 3$ and center $c = 0$:

```
sage: from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
sage: sigma = 3.0
sage: D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma)
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
>>> sigma = RealNumber('3.0')
>>> D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma)
```

We ask for 100000 samples:

```
sage: from collections import defaultdict
sage: counter = defaultdict(Integer)
sage: n = 0
sage: def add_samples(i):
....:     global counter, n
....:     for _ in range(i):
....:         counter[D()] += 1
....:         n += 1

sage: add_samples(100000)
```

```
>>> from sage.all import *
>>> from collections import defaultdict
>>> counter = defaultdict(Integer)
>>> n = Integer(0)
>>> def add_samples(i):
...     global counter, n
```

(continues on next page)

(continued from previous page)

```
...     for _ in range(i):
...         counter[D()] += Integer(1)
...         n += Integer(1)

>>> add_samples(Integer(100000))
```

These are sampled with a probability proportional to $\exp(-x^2/18)$. More precisely we have to normalise by dividing by the overall probability over all integers. We use the fact that hitting anything more than 6 standard deviations away is very unlikely and compute:

```
sage: bound = (6*sigma).floor()
sage: norm_factor = sum([exp(-x^2/(2*sigma^2)) for x in range(-bound,bound+1)])
sage: norm_factor
7.519...
```

```
>>> from sage.all import *
>>> bound = (Integer(6)*sigma).floor()
>>> norm_factor = sum([exp(-x**Integer(2)/(Integer(2)*sigma**Integer(2))) for x in␣
↪range(-bound,bound+Integer(1))])
>>> norm_factor
7.519...
```

With this normalisation factor, we can now test if our samples follow the expected distribution:

```
sage: expected = lambda x : ZZ(round(n*exp(-x^2/(2*sigma^2))/norm_factor))
sage: observed = lambda x : counter[x]

sage: add_samples(10000)
sage: while abs(observed(0)*1.0/expected(0)   - 1.0) > 5e-2: add_samples(10000)
sage: while abs(observed(4)*1.0/expected(4)   - 1.0) > 5e-2: add_samples(10000)
sage: while abs(observed(-10)*1.0/expected(-10) - 1.0) > 5e-2: add_samples(10000)  #␣
↪long time
```

```
>>> from sage.all import *
>>> expected = lambda x : ZZ(round(n*exp(-x**Integer(2)/
↪(Integer(2)*sigma**Integer(2)))/norm_factor))
>>> observed = lambda x : counter[x]

>>> add_samples(Integer(10000))
>>> while abs(observed(Integer(0))*RealNumber('1.0')/expected(Integer(0))   -␣
↪RealNumber('1.0')) > RealNumber('5e-2'): add_samples(Integer(10000))
>>> while abs(observed(Integer(4))*RealNumber('1.0')/expected(Integer(4))   -␣
↪RealNumber('1.0')) > RealNumber('5e-2'): add_samples(Integer(10000))
>>> while abs(observed(-Integer(10))*RealNumber('1.0')/expected(-Integer(10)) -␣
↪RealNumber('1.0')) > RealNumber('5e-2'): add_samples(Integer(10000))  # long time
```

We construct an instance with a larger width:

```
sage: from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
sage: sigma = 127
sage: D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma, algorithm=
```

(continues on next page)

(continued from previous page)

```
↪'uniform+online')
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
>>> sigma = Integer(127)
>>> D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma, algorithm=
↪'uniform+online')
```

ask for 100000 samples:

```
sage: from collections import defaultdict
sage: counter = defaultdict(Integer)
sage: n = 0
sage: def add_samples(i):
....:     global counter, n
....:     for _ in range(i):
....:         counter[D()] += 1
....:         n += 1

sage: add_samples(100000)
```

```
>>> from sage.all import *
>>> from collections import defaultdict
>>> counter = defaultdict(Integer)
>>> n = Integer(0)
>>> def add_samples(i):
...     global counter, n
...     for _ in range(i):
...         counter[D()] += Integer(1)
...         n += Integer(1)

>>> add_samples(Integer(100000))
```

and check if the proportions fit:

```
sage: expected = lambda x, y: (
....:     exp(-x^2/(2*sigma^2))/exp(-y^2/(2*sigma^2)).n())
sage: observed = lambda x, y: float(counter[x])/counter[y]

sage: while not all(v in counter for v in (0, 1, -100)): add_samples(10000)

sage: while abs(expected(0, 1) - observed(0, 1)) > 2e-1: add_samples(10000)
sage: while abs(expected(0, -100) - observed(0, -100)) > 2e-1: add_samples(10000)
```

```
>>> from sage.all import *
>>> expected = lambda x, y: (
...     exp(-x**Integer(2)/(Integer(2)*sigma**Integer(2)))/exp(-y**Integer(2)/
↪(Integer(2)*sigma**Integer(2))).n())
>>> observed = lambda x, y: float(counter[x])/counter[y]

>>> while not all(v in counter for v in (Integer(0), Integer(1), -Integer(100))): add_
```

(continues on next page)

(continued from previous page)

```
↪samples(Integer(10000))

>>> while abs(expected(Integer(0), Integer(1)) - observed(Integer(0), Integer(1))) >␣
↪RealNumber('2e-1'): add_samples(Integer(10000))
>>> while abs(expected(Integer(0), -Integer(100)) - observed(Integer(0), -
↪Integer(100))) > RealNumber('2e-1'): add_samples(Integer(10000))
```

We construct a sampler with $c\%1! = 0$:

```
sage: from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
sage: sigma = 3
sage: D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma, c=1/2)
sage: s = 0
sage: n = 0
sage: def add_samples(i):
....:     global s, n
....:     for _ in range(i):
....:         s += D()
....:         n += 1
....:
sage: add_samples(100000)
sage: while abs(float(s)/n - 0.5) > 5e-2: add_samples(10000)
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
>>> sigma = Integer(3)
>>> D = DiscreteGaussianDistributionIntegerSampler(sigma=sigma, c=Integer(1)/
↪Integer(2))
>>> s = Integer(0)
>>> n = Integer(0)
>>> def add_samples(i):
...     global s, n
...     for _ in range(i):
...         s += D()
...         n += Integer(1)
....:
>>> add_samples(Integer(100000))
>>> while abs(float(s)/n - RealNumber('0.5')) > RealNumber('5e-2'): add_
↪samples(Integer(10000))
```

REFERENCES:

- [DDLL2013]

class sage.stats.distributions.discrete_gaussian_integer.
DiscreteGaussianDistributionIntegerSampler

Bases: SageObject

A Discrete Gaussian Sampler using rejection sampling.

__init__(*sigma*, *c=0*, *tau=6*, *algorithm=None*, *precision='mp'*)

Construct a new sampler for a discrete Gaussian distribution.

INPUT:

- `sigma` – samples x are accepted with probability proportional to $\exp(-(x-c)^2/(2\sigma^2))$
- `c` – the mean of the distribution. The value of `c` does not have to be an integer. However, some algorithms only support integer-valued `c` (default: `0`)
- `tau` – samples outside the range $(\lfloor c \rceil - \lceil \sigma \rceil, ..., \lfloor c \rceil + \lceil \sigma \rceil)$ are considered to have probability zero. This bound applies to algorithms which sample from the uniform distribution (default: `6`)
- **`algorithm` – see list below (default: `'uniform+table'` for** σ bounded by `DiscreteGaussianDistributionIntegerSampler.table_cutoff` and `'uniform+online'` for bigger)
- `precision` – either `'mp'` for multi-precision where the actual precision used is taken from sigma or `'dp'` for double precision. In the latter case results are not reproducible. (default: `'mp'`)

ALGORITHMS:

- `'uniform+table'` – classical rejection sampling, sampling from the uniform distribution and accepted with probability proportional to $\exp(-(x-c)^2/(2\sigma^2))$ where $\exp(-(x-c)^2/(2\sigma^2))$ is precomputed and stored in a table. Any real-valued σ is supported.
- `'uniform+logtable'` – samples are drawn from a uniform distribution and accepted with probability proportional to $\exp(-(x-c)^2/(2\sigma^2))$ where $\exp(-(x-c)^2/(2\sigma^2))$ is computed using logarithmically many calls to Bernoulli distributions. See [DDLL2013] for details. Only integer-valued σ are supported.
- `'uniform+online'` – samples are drawn from a uniform distribution and accepted with probability proportional to $\exp(-(x-c)^2/(2\sigma^2))$ where $\exp(-(x-c)^2/(2\sigma^2))$ is computed in each invocation. Typically this is very slow. See [DDLL2013] for details. Any real-valued σ is accepted.
- `'sigma2+logtable'` – samples are drawn from an easily samplable distribution with $\sigma = k \cdot \sigma_2$ with $\sigma_2 = \sqrt{1/(2\log 2)}$ and accepted with probability proportional to $\exp(-(x-c)^2/(2\sigma^2))$ where $\exp(-(x-c)^2/(2\sigma^2))$ is computed using logarithmically many calls to Bernoulli distributions (but no calls to exp). See [DDLL2013] for details. Note that this sampler adjusts σ to match $k \cdot \sigma_2$ for some integer k. Only integer-valued σ are supported.

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm=
↪'uniform+online')
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm='uniform+table
↪')
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm=
↪'uniform+logtable')
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'uniform+online')
```

(continues on next page)

(continued from previous page)

```
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'uniform+table')
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'uniform+logtable')
Discrete Gaussian sampler over the Integers with sigma = 3.000000 and c = 0.
↪000000
```

Note that `'sigma2+logtable'` adjusts :

```
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm=
↪'sigma2+logtable')
Discrete Gaussian sampler over the Integers with sigma = 3.397287 and c = 0.
↪000000
```

```
>>> from sage.all import *
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'sigma2+logtable')
Discrete Gaussian sampler over the Integers with sigma = 3.397287 and c = 0.
↪000000
```

__call__()

Return a new sample.

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm=
↪'uniform+online')()  # random
-3
sage: DiscreteGaussianDistributionIntegerSampler(3.0, algorithm='uniform+table
↪')()  # random
3
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_integer import␣
↪DiscreteGaussianDistributionIntegerSampler
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'uniform+online')()  # random
-3
>>> DiscreteGaussianDistributionIntegerSampler(RealNumber('3.0'), algorithm=
↪'uniform+table')()  # random
3
```

algorithm

c

sigma

```
table_cutoff = 1000000

tau
```

CHAPTER

NINE

DISCRETE GAUSSIAN SAMPLERS FOR $\mathbf{Z}[x]$

This class realizes oracles which returns polynomials in $\mathbf{Z}[x]$ where each coefficient is sampled independently with a probability proportional to $\exp(-(x-c)^2/(2^2))$.

AUTHORS:

- Martin Albrecht, Robert Fitzpatrick, Daniel Cabracas, Florian Göpfert, Michael Schneider: initial version

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
sage: sigma = 3.0; n = 1000
sage: l = [DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 64, sigma)()
....:      for _ in range(n)]
sage: l = [vector(f).norm().n() for f in l]
sage: from numpy import mean                                                #␣
↪needs numpy
sage: mean(l), sqrt(64)*sigma  # abs tol 5e-1                               #␣
↪needs numpy
(24.0, 24.0)
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
>>> sigma = RealNumber('3.0'); n = Integer(1000)
>>> l = [DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], Integer(64), sigma)()
...      for _ in range(n)]
>>> l = [vector(f).norm().n() for f in l]
>>> from numpy import mean                                                  #␣
↪needs numpy
>>> mean(l), sqrt(Integer(64))*sigma  # abs tol 5e-1                          ␣
↪     # needs numpy
(24.0, 24.0)
```

class sage.stats.distributions.discrete_gaussian_polynomial.**DiscreteGaussianDistributionPolynomialSar**

Bases: SageObject

Discrete Gaussian sampler for polynomials.

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
sage: p = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 8, 3.0)()
sage: p.parent()
Univariate Polynomial Ring in x over Integer Ring
sage: p.degree() < 8
True
sage: gs = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 8, 3.0)
sage: [gs() for _ in range(3)]  # random
[4*x^7 + 4*x^6 - 4*x^5 + 2*x^4 + x^3 - 4*x + 7, -5*x^6 + 4*x^5 - 3*x^3 + 4*x^2 +␣
↪x, 2*x^7 + 2*x^6 + 2*x^5 - x^4 - 2*x^2 + 3*x + 1]
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
>>> p = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], Integer(8),␣
↪RealNumber('3.0'))()
>>> p.parent()
Univariate Polynomial Ring in x over Integer Ring
>>> p.degree() < Integer(8)
True
>>> gs = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], Integer(8),␣
↪RealNumber('3.0'))
>>> [gs() for _ in range(Integer(3))]  # random
[4*x^7 + 4*x^6 - 4*x^5 + 2*x^4 + x^3 - 4*x + 7, -5*x^6 + 4*x^5 - 3*x^3 + 4*x^2 +␣
↪x, 2*x^7 + 2*x^6 + 2*x^5 - x^4 - 2*x^2 + 3*x + 1]
```

__init__(*P*, *n*, *sigma*)

Construct a sampler for univariate polynomials of degree `n-1` where coefficients are drawn independently with standard deviation `sigma`.

INPUT:

- `P` – a univariate polynomial ring over the Integers
- `n` – number of coefficients to be sampled
- `sigma` – coefficients x are accepted with probability proportional to $\exp(-x^2/(2^2))$. If an object of type *sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler* is passed, then this sampler is used to sample coefficients.

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
sage: p = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 8, 3.0)()
sage: p.parent()
Univariate Polynomial Ring in x over Integer Ring
sage: p.degree() < 8
True
sage: gs = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 8, 3.0)
sage: [gs() for _ in range(3)]  # random
[4*x^7 + 4*x^6 - 4*x^5 + 2*x^4 + x^3 - 4*x + 7, -5*x^6 + 4*x^5 - 3*x^3 + 4*x^
↪2 + x, 2*x^7 + 2*x^6 + 2*x^5 - x^4 - 2*x^2 + 3*x + 1]
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
>>> p = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], Integer(8),␣
↪RealNumber('3.0'))()
>>> p.parent()
Univariate Polynomial Ring in x over Integer Ring
>>> p.degree() < Integer(8)
True
>>> gs = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], Integer(8),␣
↪RealNumber('3.0'))
>>> [gs() for _ in range(Integer(3))]  # random
[4*x^7 + 4*x^6 - 4*x^5 + 2*x^4 + x^3 - 4*x + 7, -5*x^6 + 4*x^5 - 3*x^3 + 4*x^
↪2 + x, 2*x^7 + 2*x^6 + 2*x^5 - x^4 - 2*x^2 + 3*x + 1]
```

__call__()

Return a new sample.

EXAMPLES:

```
sage: from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
sage: sampler = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'], 8, 12.
↪0)
sage: sampler().parent()
Univariate Polynomial Ring in x over Integer Ring
sage: sampler().degree() <= 7
True
```

```
>>> from sage.all import *
>>> from sage.stats.distributions.discrete_gaussian_polynomial import␣
↪DiscreteGaussianDistributionPolynomialSampler
>>> sampler = DiscreteGaussianDistributionPolynomialSampler(ZZ['x'],␣
↪Integer(8), RealNumber('12.0'))
>>> sampler().parent()
Univariate Polynomial Ring in x over Integer Ring
>>> sampler().degree() <= Integer(7)
True
```

CHAPTER

TEN

DISCRETE GAUSSIAN SAMPLERS OVER LATTICES

This file implements oracles which return samples from a lattice following a discrete Gaussian distribution. That is, if σ is big enough relative to the provided basis, then vectors are returned with a probability proportional to $\exp(-|x - c|_2^2/(2\sigma^2))$. More precisely lattice vectors in $x \in \Lambda$ are returned with probability:

$$\frac{\exp(-|x - c|_2^2/(2\sigma^2))}{\sum_{v\in\Lambda} \exp(-|v|_2^2/(2\sigma^2))}.$$

AUTHORS:

- Martin Albrecht (2014-06-28): initial version
- Gareth Ma (2023-09-22): implement non-spherical sampling

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^10, 3.0)
sage: D(), D(), D()  # random
((3, 0, -5, 0, -1, -3, 3, 3, -7, 2), (4, 0, 1, -2, -4, -4, 4, 0, 1, -4), (-3, 0, 4, 5,
↪ 0, 1, 3, 2, 0, -1))
sage: a = D()
sage: a.parent()
Ambient free module of rank 10 over the principal ideal domain Integer Ring
```

```
>>> from sage.all import *
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(10),␣
↪RealNumber('3.0'))
>>> D(), D(), D()  # random
((3, 0, -5, 0, -1, -3, 3, 3, -7, 2), (4, 0, 1, -2, -4, -4, 4, 0, 1, -4), (-3, 0, 4, 5,
↪ 0, 1, 3, 2, 0, -1))
>>> a = D()
>>> a.parent()
Ambient free module of rank 10 over the principal ideal domain Integer Ring
```

class sage.stats.distributions.discrete_gaussian_lattice.**DiscreteGaussianDistributionLatticeSampler**(

Bases: SageObject

GPV sampler for Discrete Gaussians over Lattices.

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^10, 3.0); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(0, 0,␣
↪0, 0, 0, 0, 0, 0, 0, 0) over lattice with basis

[1 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 1]
```

```
>>> from sage.all import *
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(10),␣
↪RealNumber('3.0')); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(0, 0,␣
↪0, 0, 0, 0, 0, 0, 0, 0) over lattice with basis
<BLANKLINE>
[1 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 1]
```

We plot a histogram:

```
sage: import warnings
sage: warnings.simplefilter('ignore', UserWarning)
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(identity_
↪matrix(2), 3.0)
sage: S = [D() for _ in range(2^12)]
sage: l = [vector(v.list() + [S.count(v)]) for v in set(S)]
sage: list_plot3d(l, point_list=True, interpolation='nn')
Graphics3d Object
```

```
>>> from sage.all import *
>>> import warnings
>>> warnings.simplefilter('ignore', UserWarning)
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(identity_
↪matrix(Integer(2)), RealNumber('3.0'))
>>> S = [D() for _ in range(Integer(2)**Integer(12))]
>>> l = [vector(v.list() + [S.count(v)]) for v in set(S)]
```

(continues on next page)

(continued from previous page)

```
>>> list_plot3d(l, point_list=True, interpolation='nn')
Graphics3d Object
```

REFERENCES:

- [GPV2008]

__init__(*B*, *sigma=1*, *c=0*, *r=None*, *precision=None*, *sigma_basis=False*)

Construct a discrete Gaussian sampler over the lattice $\Lambda(B)$ with parameter `sigma` and center c.

INPUT:

- `B` – a (row) basis for the lattice, one of the following:
  - an integer matrix,
  - an object with a `.matrix()` method, e.g. `ZZ^n`, or
  - an object where `matrix(B)` succeeds, e.g. a list of vectors
- `sigma` – Gaussian parameter, one of the following:
  - a real number $\sigma > 0$ (spherical),
  - a positive definite matrix Σ (non-spherical), or
  - any matrix-like `S`, equivalent to $\Sigma = SS^T$, when `sigma_basis` is set
- `c` – (default: 0) center c, any vector in $\mathbf{Z}^n$ is supported, but $c \in \Lambda(B)$ is faster
- `r` – (default: `None`) rounding parameter r as defined in [Pei2010]; ignored for spherical Gaussian parameter; if not provided, set to be the maximal possible such that $\Sigma - rBB^T$ is positive definite
- `precision` – bit precision ≥ 53
- **`sigma_basis` – boolean (default: `False`); when set, `sigma` is treated as** a (row) basis, i.e. the covariance matrix is computed by $\Sigma = SS^T$

Todo

Rename class methods like `.f` and hide most of them (at least behind something like `.data`).

EXAMPLES:

```
sage: n = 2; sigma = 3.0
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^n,␣
↪sigma)
sage: f = D.f
sage: nf = D._normalisation_factor_zz(); nf
56.5486677646...

sage: from collections import defaultdict
sage: counter = defaultdict(Integer); m = 0
sage: def add_samples(i):
....:     global counter, m
....:     for _ in range(i):
....:         counter[D()] += 1
....:         m += 1
```

(continues on next page)

(continued from previous page)

```
sage: v = vector(ZZ, n, (-3, -3))
sage: v.set_immutable()
sage: while v not in counter: add_samples(1000)
sage: while abs(m*f(v)*1.0/nf/counter[v] - 1.0) >= 0.1:
....:     add_samples(1000)

sage: counter = defaultdict(Integer); m = 0
sage: v = vector(ZZ, n, (0, 0))
sage: v.set_immutable()
sage: while v not in counter:
....:     add_samples(1000)
sage: while abs(m*f(v)*1.0/nf/counter[v] - 1.0) >= 0.1:
....:     add_samples(1000)
```

```
>>> from sage.all import *
>>> n = Integer(2); sigma = RealNumber('3.0')
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**n, sigma)
>>> f = D.f
>>> nf = D._normalisation_factor_zz(); nf
56.5486677646...

>>> from collections import defaultdict
>>> counter = defaultdict(Integer); m = Integer(0)
>>> def add_samples(i):
...     global counter, m
...     for _ in range(i):
...         counter[D()] += Integer(1)
...         m += Integer(1)

>>> v = vector(ZZ, n, (-Integer(3), -Integer(3)))
>>> v.set_immutable()
>>> while v not in counter: add_samples(Integer(1000))
>>> while abs(m*f(v)*RealNumber('1.0')/nf/counter[v] - RealNumber('1.0')) >=␣
↪RealNumber('0.1'):
...     add_samples(Integer(1000))

>>> counter = defaultdict(Integer); m = Integer(0)
>>> v = vector(ZZ, n, (Integer(0), Integer(0)))
>>> v.set_immutable()
>>> while v not in counter:
...     add_samples(Integer(1000))
>>> while abs(m*f(v)*RealNumber('1.0')/nf/counter[v] - RealNumber('1.0')) >=␣
↪RealNumber('0.1'):
...     add_samples(Integer(1000))
```

Spherical covariance are automatically handled:

```
sage: distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3,␣
↪sigma=Matrix(3, 3, 2))
Discrete Gaussian sampler with Gaussian parameter σ = 2.00000000000000, c=(0,␣
↪0, 0) over lattice with basis
```

(continues on next page)

(continued from previous page)

```
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3),␣
↪sigma=Matrix(Integer(3), Integer(3), Integer(2)))
Discrete Gaussian sampler with Gaussian parameter σ = 2.00000000000000, c=(0,␣
↪0, 0) over lattice with basis
<BLANKLINE>
[1 0 0]
[0 1 0]
[0 0 1]
```

The sampler supports non-spherical covariance in the form of a Gram matrix:

```
sage: n = 3
sage: Sigma = Matrix(ZZ, [[5, -2, 4], [-2, 10, -5], [4, -5, 5]])
sage: c = vector(ZZ, [7, 2, 5])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^n,␣
↪Sigma, c)
sage: f = D.f
sage: # This has not been properly implemented...
sage: nf = D._normalisation_factor_zz(); nf  # long time
78.6804...
```

```
>>> from sage.all import *
>>> n = Integer(3)
>>> Sigma = Matrix(ZZ, [[Integer(5), -Integer(2), Integer(4)], [-Integer(2),␣
↪Integer(10), -Integer(5)], [Integer(4), -Integer(5), Integer(5)]])
>>> c = vector(ZZ, [Integer(7), Integer(2), Integer(5)])
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**n, Sigma,
↪ c)
>>> f = D.f
>>> # This has not been properly implemented...
>>> nf = D._normalisation_factor_zz(); nf  # long time
78.6804...
```

We can compute the expected number of samples before sampling a vector:

```
sage: v = vector(ZZ, n, (11, 4, 8))
sage: v.set_immutable()
sage: 1 / (f(v) / nf)  # long time
2553.9461...

sage: counter = defaultdict(Integer); m = 0
sage: while v not in counter:
....:     add_samples(1000)
sage: sum(counter.values())  # random
3000
sage: while abs(m*f(v)*1.0/nf/counter[v] - 1.0) >= 0.1:  # long time
```

(continues on next page)

(continued from previous page)

```
....:     add_samples(1000)
```

```
>>> from sage.all import *
>>> v = vector(ZZ, n, (Integer(11), Integer(4), Integer(8)))
>>> v.set_immutable()
>>> Integer(1) / (f(v) / nf)  # long time
2553.9461...

>>> counter = defaultdict(Integer); m = Integer(0)
>>> while v not in counter:
...     add_samples(Integer(1000))
>>> sum(counter.values())  # random
3000
>>> while abs(m*f(v)*RealNumber('1.0')/nf/counter[v] - RealNumber('1.0')) >=␣
↪RealNumber('0.1'):  # long time
...     add_samples(Integer(1000))
```

If the covariance provided is not positive definite, an error is thrown:

```
sage: Sigma = Matrix(ZZ, [[0, 1], [1, 0]])
sage: distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^2, Sigma)
Traceback (most recent call last):
...
RuntimeError: Sigma(=[0.000000000000000  1.00000000000000]
[ 1.00000000000000 0.000000000000000]) is not positive definite
```

```
>>> from sage.all import *
>>> Sigma = Matrix(ZZ, [[Integer(0), Integer(1)], [Integer(1), Integer(0)]])
>>> distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(2),␣
↪Sigma)
Traceback (most recent call last):
...
RuntimeError: Sigma(=[0.000000000000000  1.00000000000000]
[ 1.00000000000000 0.000000000000000]) is not positive definite
```

The sampler supports passing a basis for the covariance:

```
sage: n = 3
sage: S = Matrix(ZZ, [[2, 0, 0], [-1, 3, 0], [2, -1, 1]])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^n, S,␣
↪sigma_basis=True)
sage: D.sigma()
[ 4.00000000000000 -2.00000000000000  4.00000000000000]
[-2.00000000000000  10.0000000000000 -5.00000000000000]
[ 4.00000000000000 -5.00000000000000  6.00000000000000]
```

```
>>> from sage.all import *
>>> n = Integer(3)
>>> S = Matrix(ZZ, [[Integer(2), Integer(0), Integer(0)], [-Integer(1),␣
↪Integer(3), Integer(0)], [Integer(2), -Integer(1), Integer(1)]])
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ**n, S,␣
↪sigma_basis=True)
```

(continues on next page)

(continued from previous page)

```
>>> D.sigma()
[ 4.00000000000000 -2.00000000000000  4.00000000000000]
[-2.00000000000000  10.0000000000000 -5.00000000000000]
[ 4.00000000000000 -5.00000000000000  6.00000000000000]
```

The non-spherical sampler supports offline computation to speed up sampling. This will be useful when changing the center c is supported. The difference is more significant for larger matrices. For 128x128 we observe a 4x speedup (86s -> 20s):

```
sage: D.offline_samples = []
sage: T = 2**12
sage: L = [D() for _ in range(T)] # 560ms
sage: D.add_offline_samples(T)    # 150ms
sage: L = [D() for _ in range(T)] # 370ms
```

```
>>> from sage.all import *
>>> D.offline_samples = []
>>> T = Integer(2)**Integer(12)
>>> L = [D() for _ in range(T)] # 560ms
>>> D.add_offline_samples(T)    # 150ms
>>> L = [D() for _ in range(T)] # 370ms
```

We can also initialise with matrix-like objects:

```
sage: qf = matrix(3, [2, 1, 1,  1, 2, 1,  1, 1, 2])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(qf, 3.0); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(0,␣
↪0, 0) over lattice with basis

[2 1 1]
[1 2 1]
[1 1 2]
sage: D().parent() is D.c().parent()
True
```

```
>>> from sage.all import *
>>> qf = matrix(Integer(3), [Integer(2), Integer(1), Integer(1),  Integer(1),␣
↪Integer(2), Integer(1),  Integer(1), Integer(1), Integer(2)])
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(qf,␣
↪RealNumber('3.0')); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(0,␣
↪0, 0) over lattice with basis
<BLANKLINE>
[2 1 1]
[1 2 1]
[1 1 2]
>>> D().parent() is D.c().parent()
True
```

__call__()

Return a new sample.

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3, 3.0,␣
↪c=(1,0,0))
sage: L = [D() for _ in range(2^12)]
sage: mean_L = sum(L) / len(L)
sage: norm(mean_L.n() - D.c()) < 0.25
True

sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3, 3.0,␣
↪c=(1/2,0,0))
sage: L = [D() for _ in range(2^12)]     # long time
sage: mean_L = sum(L) / len(L)           # long time
sage: norm(mean_L.n() - D.c()) < 0.25    # long time
True

sage: import numpy
sage: M = matrix(ZZ, [[1,2],[0,1]])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(M, 20.0)
sage: L = [D() for _ in range(2^12)]                                          ␣
↪     # long time
sage: div = numpy.mean([abs(x) for x,y in L]) / numpy.mean([abs(y) for x,y,␣
↪in L]) # long time
sage: 0.9 < div < 1.1                                                        ␣
↪     # long time
True
```

```
>>> from sage.all import *
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), RealNumber('3.0
↪'), c=(Integer(1),Integer(0),Integer(0)))
>>> L = [D() for _ in range(Integer(2)**Integer(12))]
>>> mean_L = sum(L) / len(L)
>>> norm(mean_L.n() - D.c()) < RealNumber('0.25')
True

>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), RealNumber('3.0
↪'), c=(Integer(1)/Integer(2),Integer(0),Integer(0)))
>>> L = [D() for _ in range(Integer(2)**Integer(12))]    # long time
>>> mean_L = sum(L) / len(L)          # long time
>>> norm(mean_L.n() - D.c()) < RealNumber('0.25')    # long time
True

>>> import numpy
>>> M = matrix(ZZ, [[Integer(1),Integer(2)],[Integer(0),Integer(1)]])
>>> D = distributions.DiscreteGaussianDistributionLatticeSampler(M,␣
↪RealNumber('20.0'))
>>> L = [D() for _ in range(Integer(2)**Integer(12))]                         ␣
↪                     # long time
>>> div = numpy.mean([abs(x) for x,y in L]) / numpy.mean([abs(y) for x,y, in␣
↪L]) # long time
>>> RealNumber('0.9') < div < RealNumber('1.1')                              ␣
↪                                # long time
True
```

add_offline_samples(*cnt=1*)

Precompute samples from $B^{-1}D_1$ to be used in `_call_non_spherical`.

EXAMPLES:

```
sage: Sigma = Matrix([[5, -2, 4], [-2, 10, -5], [4, -5, 5]])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3,␣
↪Sigma)
sage: assert not D.is_spherical
sage: D.add_offline_samples(2^12)
sage: L = [D() for _ in range(2^12)] # Takes less time
```

```
>>> from sage.all import *
>>> Sigma = Matrix([[Integer(5), -Integer(2), Integer(4)], [-Integer(2),␣
↪Integer(10), -Integer(5)], [Integer(4), -Integer(5), Integer(5)]])
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), Sigma)
>>> assert not D.is_spherical
>>> D.add_offline_samples(Integer(2)**Integer(12))
>>> L = [D() for _ in range(Integer(2)**Integer(12))] # Takes less time
```

c()

Center c.

Samples from this sampler will be centered at c.

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3, 3.0,␣
↪c=(1,0,0)); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(1,␣
↪0, 0) over lattice with basis

[1 0 0]
[0 1 0]
[0 0 1]

sage: D.c()
(1, 0, 0)
```

```
>>> from sage.all import *
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), RealNumber('3.0
↪'), c=(Integer(1),Integer(0),Integer(0))); D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(1,␣
↪0, 0) over lattice with basis
<BLANKLINE>
[1 0 0]
[0 1 0]
[0 0 1]

>>> D.c()
(1, 0, 0)
```

static compute_precision(*precision*, *sigma*)

Compute precision to use.

INPUT:

- `precision` – integer >= 53 nor `None`
- `sigma` – if `precision` is `None` then the precision of `sigma` is used

EXAMPLES:

```
sage: DGL = distributions.DiscreteGaussianDistributionLatticeSampler
sage: DGL.compute_precision(100, RR(3))
100
sage: DGL.compute_precision(100, RealField(200)(3))
100
sage: DGL.compute_precision(100, 3)
100
sage: DGL.compute_precision(None, RR(3))
53
sage: DGL.compute_precision(None, RealField(200)(3))
200
sage: DGL.compute_precision(None, 3)
53
```

```
>>> from sage.all import *
>>> DGL = distributions.DiscreteGaussianDistributionLatticeSampler
>>> DGL.compute_precision(Integer(100), RR(Integer(3)))
100
>>> DGL.compute_precision(Integer(100), RealField(Integer(200))(Integer(3)))
100
>>> DGL.compute_precision(Integer(100), Integer(3))
100
>>> DGL.compute_precision(None, RR(Integer(3)))
53
>>> DGL.compute_precision(None, RealField(Integer(200))(Integer(3)))
200
>>> DGL.compute_precision(None, Integer(3))
53
```

f(*x*)

Compute the Gaussian $\rho_{\Lambda,c,\Sigma}$.

EXAMPLES:

```
sage: Sigma = Matrix(ZZ, [[5, -2, 4], [-2, 10, -5], [4, -5, 5]])
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3,␣
↪Sigma)
sage: D.f([1, 0, 1])
0.802518797962478
sage: D.f([1, 0, 3])
0.00562800641440405
```

```
>>> from sage.all import *
>>> Sigma = Matrix(ZZ, [[Integer(5), -Integer(2), Integer(4)], [-Integer(2),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(10), -Integer(5)], [Integer(4), -Integer(5), Integer(5)]])
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), Sigma)
>>> D.f([Integer(1), Integer(0), Integer(1)])
0.802518797962478
>>> D.f([Integer(1), Integer(0), Integer(3)])
0.00562800641440405
```

set_c(*c*)

Modifies center *c*.

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3, 3.0,␣
↪c=(1,0,0))
sage: D.set_c((2, 0, 0))
sage: D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(2,␣
↪0, 0) over lattice with basis

[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), RealNumber('3.0
↪'), c=(Integer(1),Integer(0),Integer(0)))
>>> D.set_c((Integer(2), Integer(0), Integer(0)))
>>> D
Discrete Gaussian sampler with Gaussian parameter σ = 3.00000000000000, c=(2,␣
↪0, 0) over lattice with basis
<BLANKLINE>
[1 0 0]
[0 1 0]
[0 0 1]
```

sigma()

Gaussian parameter σ.

If σ is a real number, samples from this sampler will have expected norm $\sqrt{n}\sigma$ where n is the dimension of the lattice.

EXAMPLES:

```
sage: D = distributions.DiscreteGaussianDistributionLatticeSampler(ZZ^3, 3.0,␣
↪c=(1,0,0))
sage: D.sigma()
3.00000000000000
```

```
>>> from sage.all import *
>>> D = distributions.
↪DiscreteGaussianDistributionLatticeSampler(ZZ**Integer(3), RealNumber('3.0
```

(continues on next page)

(continued from previous page)

```
↪'), c=(Integer(1),Integer(0),Integer(0)))
>>> D.sigma()
3.00000000000000
```

CHAPTER

ELEVEN

T-TEST USING R

sage.stats.r.**ttest**(*x*, *y*, *conf_level=0.95*, ***kw*)

T-Test using R.

INPUT:

- `x`, `y` – vectors of same length
- `conf_level` – confidence level of the interval, [0,1) in percent

OUTPUT:

Tuple: (p-value, R return object)

EXAMPLES:

```
sage: a, b = ttest([1,2,3,4,5],[1,2,3,3.5,5.121]); a  # abs tol 1e-12  # optional␣
↪- rpy2
0.9410263720274274
```

```
>>> from sage.all import *
>>> a, b = ttest([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)],
↪[Integer(1),Integer(2),Integer(3),RealNumber('3.5'),RealNumber('5.121')]); a  #␣
↪abs tol 1e-12  # optional - rpy2
0.9410263720274274
```

CHAPTER

TWELVE

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

S

sage.stats.basic_stats, 1
sage.stats.distributions.discrete_gaussian_integer, 93
sage.stats.distributions.discrete_gaussian_lattice, 105
sage.stats.distributions.discrete_gaussian_polynomial, 101
sage.stats.hmm.chmm, 65
sage.stats.hmm.distributions, 81
sage.stats.hmm.hmm, 49
sage.stats.hmm.util, 89
sage.stats.intlist, 11
sage.stats.r, 117
sage.stats.time_series, 17

INDEX

Non-alphabetical

__call__() (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler method*), 98
__call__() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 111
__call__() (*sage.stats.distributions.discrete_gaussian_polynomial.DiscreteGaussianDistributionPolynomialSampler method*), 103
__init__() (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler method*), 96
__init__() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 107
__init__() (*sage.stats.distributions.discrete_gaussian_polynomial.DiscreteGaussianDistributionPolynomialSampler method*), 102

A

abs() (*sage.stats.time_series.TimeSeries method*), 19
add_entries() (*sage.stats.time_series.TimeSeries method*), 19
add_offline_samples() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 112
add_scalar() (*sage.stats.time_series.TimeSeries method*), 20
algorithm (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler attribute*), 98
autocorrelation() (*sage.stats.time_series.TimeSeries method*), 20
autocovariance() (*sage.stats.time_series.TimeSeries method*), 21
autoregressive_fit() (*in module sage.stats.time_series*), 47
autoregressive_fit() (*sage.stats.time_series.TimeSeries method*), 23
autoregressive_forecast() (*sage.stats.time_series.TimeSeries method*), 24

B

baum_welch() (*sage.stats.hmm.chmm.GaussianHiddenMarkovModel method*), 68
baum_welch() (*sage.stats.hmm.chmm.GaussianMixtureHiddenMarkovModel method*), 75
baum_welch() (*sage.stats.hmm.hmm.DiscreteHiddenMarkovModel method*), 51

C

c (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler attribute*), 98
c() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 113
central_moment() (*sage.stats.time_series.TimeSeries method*), 25
clip_remove() (*sage.stats.time_series.TimeSeries method*), 26
compute_precision() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler static method*), 113
correlation() (*sage.stats.time_series.TimeSeries method*), 26
covariance() (*sage.stats.time_series.TimeSeries method*), 27

D

diffs() (*sage.stats.time_series.TimeSeries method*), 27
DiscreteDistribution (*class in sage.stats.hmm.distributions*), 81
DiscreteGaussianDistributionIntegerSampler (*class in sage.stats.distributions.discrete_gaussian_integer*), 96
DiscreteGaussianDistributionLatticeSampler (*class in sage.stats.distributions.discrete_gaussian_lattice*), 105
DiscreteGaussianDistributionPolynomialSampler (*class in sage.stats.distributions.discrete_gaussian_polynomial*), 101
DiscreteHiddenMarkovModel (*class in sage.stats.hmm.hmm*), 49

Distribution (*class in sage.stats.hmm.distributions*), 81

E

emission_matrix() (*sage.stats.hmm.hmm.DiscreteHiddenMarkovModel method*), 53
emission_parameters() (*sage.stats.hmm.chmm.GaussianHiddenMarkovModel method*), 71
emission_parameters() (*sage.stats.hmm.chmm.GaussianMixtureHiddenMarkovModel method*), 78
exp() (*sage.stats.time_series.TimeSeries method*), 27
exponential_moving_average() (*sage.stats.time_series.TimeSeries method*), 28
extend() (*sage.stats.time_series.TimeSeries method*), 29

F

f() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 114
fft() (*sage.stats.time_series.TimeSeries method*), 29
fix() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 83

G

GaussianDistribution (*class in sage.stats.hmm.distributions*), 82
GaussianHiddenMarkovModel (*class in sage.stats.hmm.chmm*), 65
GaussianMixtureDistribution (*class in sage.stats.hmm.distributions*), 82
GaussianMixtureHiddenMarkovModel (*class in sage.stats.hmm.chmm*), 75
generate_sequence() (*sage.stats.hmm.chmm.GaussianHiddenMarkovModel method*), 71
generate_sequence() (*sage.stats.hmm.hmm.DiscreteHiddenMarkovModel method*), 54
graph() (*sage.stats.hmm.hmm.HiddenMarkovModel method*), 58

H

HiddenMarkovModel (*class in sage.stats.hmm.hmm*), 58
histogram() (*sage.stats.time_series.TimeSeries method*), 30
HMM_Util (*class in sage.stats.hmm.util*), 89
hurst_exponent() (*sage.stats.time_series.TimeSeries method*), 31

I

ifft() (*sage.stats.time_series.TimeSeries method*), 31
initial_probabilities() (*sage.stats.hmm.hmm.HiddenMarkovModel method*), 59
initial_probs_to_TimeSeries() (*sage.stats.hmm.util.HMM_Util method*), 89
IntList (*class in sage.stats.intlist*), 11
is_fixed() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 84

L

list() (*sage.stats.intlist.IntList method*), 11
list() (*sage.stats.time_series.TimeSeries method*), 32
log() (*sage.stats.time_series.TimeSeries method*), 32
log_likelihood() (*sage.stats.hmm.chmm.GaussianHiddenMarkovModel method*), 73
log_likelihood() (*sage.stats.hmm.hmm.DiscreteHiddenMarkovModel method*), 55

M

max() (*sage.stats.intlist.IntList method*), 11
max() (*sage.stats.time_series.TimeSeries method*), 33
mean() (*in module sage.stats.basic_stats*), 1
mean() (*sage.stats.time_series.TimeSeries method*), 33
median() (*in module sage.stats.basic_stats*), 2
min() (*sage.stats.intlist.IntList method*), 12
min() (*sage.stats.time_series.TimeSeries method*), 34
mode() (*in module sage.stats.basic_stats*), 3
module
    sage.stats.basic_stats, 1
    sage.stats.distributions.discrete_gaussian_integer, 93
    sage.stats.distributions.discrete_gaussian_lattice, 105
    sage.stats.distributions.discrete_gaussian_polynomial, 101
    sage.stats.hmm.chmm, 65
    sage.stats.hmm.distributions, 81
    sage.stats.hmm.hmm, 49
    sage.stats.hmm.util, 89
    sage.stats.intlist, 11
    sage.stats.r, 117
    sage.stats.time_series, 17
moment() (*sage.stats.time_series.TimeSeries method*), 34
moving_average() (*in module sage.stats.basic_stats*), 5

N

normalize_probability_TimeSeries() (*sage.stats.hmm.util.HMM_Util method*), 89
numpy() (*sage.stats.time_series.TimeSeries method*), 35

P

plot() (*sage.stats.hmm.distributions.Distribution method*), 81
plot() (*sage.stats.intlist.IntList method*), 12
plot() (*sage.stats.time_series.TimeSeries method*), 36
plot_candlestick() (*sage.stats.time_series.TimeSeries method*), 37
plot_histogram() (*sage.stats.intlist.IntList method*), 13
plot_histogram() (*sage.stats.time_series.TimeSeries method*), 37

pow() (*sage.stats.time_series.TimeSeries method*), 38
prob() (*sage.stats.hmm.distributions.Distribution method*), 81
prob() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 84
prob_m() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 85
prod() (*sage.stats.intlist.IntList method*), 13
prod() (*sage.stats.time_series.TimeSeries method*), 38

R

randomize() (*sage.stats.time_series.TimeSeries method*), 38
range_statistic() (*sage.stats.time_series.TimeSeries method*), 41
rescale() (*sage.stats.time_series.TimeSeries method*), 42
reversed() (*sage.stats.time_series.TimeSeries method*), 42

S

sage.stats.basic_stats
    module, 1
sage.stats.distributions.discrete_gaussian_integer
    module, 93
sage.stats.distributions.discrete_gaussian_lattice
    module, 105
sage.stats.distributions.discrete_gaussian_polynomial
    module, 101
sage.stats.hmm.chmm
    module, 65
sage.stats.hmm.distributions
    module, 81
sage.stats.hmm.hmm
    module, 49
sage.stats.hmm.util
    module, 89
sage.stats.intlist
    module, 11
sage.stats.r
    module, 117
sage.stats.time_series
    module, 17
sample() (*sage.stats.hmm.distributions.Distribution method*), 82
sample() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 85
sample() (*sage.stats.hmm.hmm.HiddenMarkovModel method*), 60
scale() (*sage.stats.time_series.TimeSeries method*), 42
scale_time() (*sage.stats.time_series.TimeSeries method*), 43
set_c() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 115
show() (*sage.stats.time_series.TimeSeries method*), 44
sigma (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler attribute*), 98
sigma() (*sage.stats.distributions.discrete_gaussian_lattice.DiscreteGaussianDistributionLatticeSampler method*), 115
simple_moving_average() (*sage.stats.time_series.TimeSeries method*), 44
standard_deviation() (*sage.stats.time_series.TimeSeries method*), 45
state_matrix_to_TimeSeries() (*sage.stats.hmm.util.HMM_Util method*), 90
std() (*in module sage.stats.basic_stats*), 6
sum() (*sage.stats.intlist.IntList method*), 14
sum() (*sage.stats.time_series.TimeSeries method*), 45
sums() (*sage.stats.time_series.TimeSeries method*), 46

T

table_cutoff (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler attribute*), 98
tau (*sage.stats.distributions.discrete_gaussian_integer.DiscreteGaussianDistributionIntegerSampler attribute*), 99
time_series() (*sage.stats.intlist.IntList method*), 14
TimeSeries (*class in sage.stats.time_series*), 18
transition_matrix() (*sage.stats.hmm.hmm.HiddenMarkovModel method*), 61
ttest() (*in module sage.stats.r*), 117

U

unfix() (*sage.stats.hmm.distributions.GaussianMixtureDistribution method*), 87
unpickle_discrete_hmm_v0() (*in module sage.stats.hmm.hmm*), 63
unpickle_discrete_hmm_v1() (*in module sage.stats.hmm.hmm*), 63
unpickle_gaussian_hmm_v0() (*in module sage.stats.hmm.chmm*), 78
unpickle_gaussian_hmm_v1() (*in module sage.stats.hmm.chmm*), 79
unpickle_gaussian_mixture_distribution_v1() (*in module sage.stats.hmm.distributions*), 87
unpickle_gaussian_mixture_hmm_v1() (*in module sage.stats.hmm.chmm*), 79
unpickle_intlist_v1() (*in module sage.stats.intlist*), 14
unpickle_time_series_v1() (*in module sage.stats.time_series*), 47

V

variance() (*in module sage.stats.basic_stats*), 8
variance() (*sage.stats.time_series.TimeSeries method*), 46
vector() (*sage.stats.time_series.TimeSeries method*), 46
viterbi() (*sage.stats.hmm.chmm.GaussianHiddenMarkovModel method*), 74
viterbi() (*sage.stats.hmm.hmm.DiscreteHiddenMarkovModel method*), 57